Projekt zaliczeniowy – Analiza szeregów czasowych

Anna Zgrzebna
I rok Analizy Danych

Wprowadzenie

Celem projektu jest przeprowadzenie w miarę pełnej analizy szeregu czasowego, zgodnie z zakresem materiału omawianego podczas zajęć. Analiza zostanie przeprowadzona w następujących krokach:
1. Wczytanie danych i konwersja danych na odpowiedni format,
2. Wykres szeregu czasowego, analiza trendu, wykresy sezonowe, analiza wahań sezonowych,
3. Wykresy ACF, PACF,
4. Wykonanie niezbędnych przekształceń,
5. Wygładzanie szeregu czasowego,
6. Dekompozycja szeregu czasowego,
7. Analiza widmowa,
8. Dopasowanie modeli z estymacją parametrów,
9. Wykonanie badań diagnostycznych,
10. Prognozowanie: konstrukcja prognoz punktowych i przedziałów predykcyjnych, horyzont prognozy,
11. Ocena dokładności prognoz.

Analizowanym szeregiem jest „milk_ts” z pakietu „timeSeriesDataSets”. Dane są przedstawione w formie szeregu czasowego od stycznia 1962 roku do grudnia 1975 roku. Zawiera on informację o miesięcznej produkcji mleka na krowę (w funtach) . Poniższa tabelka prezentuje omawiane dane.

	Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec
1962	589	561	640	656	727	697	640	599	568	577	553	582
1963	600	566	653	673	742	716	660	617	583	587	565	598
1964	628	618	688	705	770	736	678	639	604	611	594	634
1965	658	622	709	722	782	756	702	653	615	621	602	635
1966	677	635	736	755	811	798	735	697	661	667	645	688
1967	713	667	762	784	837	817	767	722	681	687	660	698
1968	717	696	775	796	858	826	783	740	701	706	677	711
1969	734	690	785	805	871	845	801	764	725	723	690	734
1970	750	707	807	824	886	859	819	783	740	747	711	751
1971	804	756	860	878	942	913	869	834	790	800	763	800
1972	826	799	890	900	961	935	894	855	809	810	766	805
1973	821	773	883	898	957	924	881	837	784	791	760	802
1974	828	778	889	902	969	947	908	867	815	812	773	813
1975	834	782	892	903	966	937	896	858	817	827	797	843

1. Wczytanie danych i konwersja danych na odpowiedni format.

Na samym początku wczytujemy pakiet i dane.

```
> install.packages("timeSeriesDataSets")  # Instalujemy pakiet z danymi
> library(timeSeriesDataSets)
> data(milk_ts)
> milk_ts
```

W wyniku wywołania poleceń otrzymujemy wartości pokazane w tabeli powyżej.
Wykorzystując polecenie *is.ts* sprawdzamy czy rzeczywiście mamy do czynienia z danymi w formie szeregu czasowego.

```
> is.ts(milk_ts)
[1] TRUE
```

Jako wynik otrzymujemy wartość *TRUE*. Dane mają odpowiedni format i nie wymagają przekształcenia w szereg czasowy.

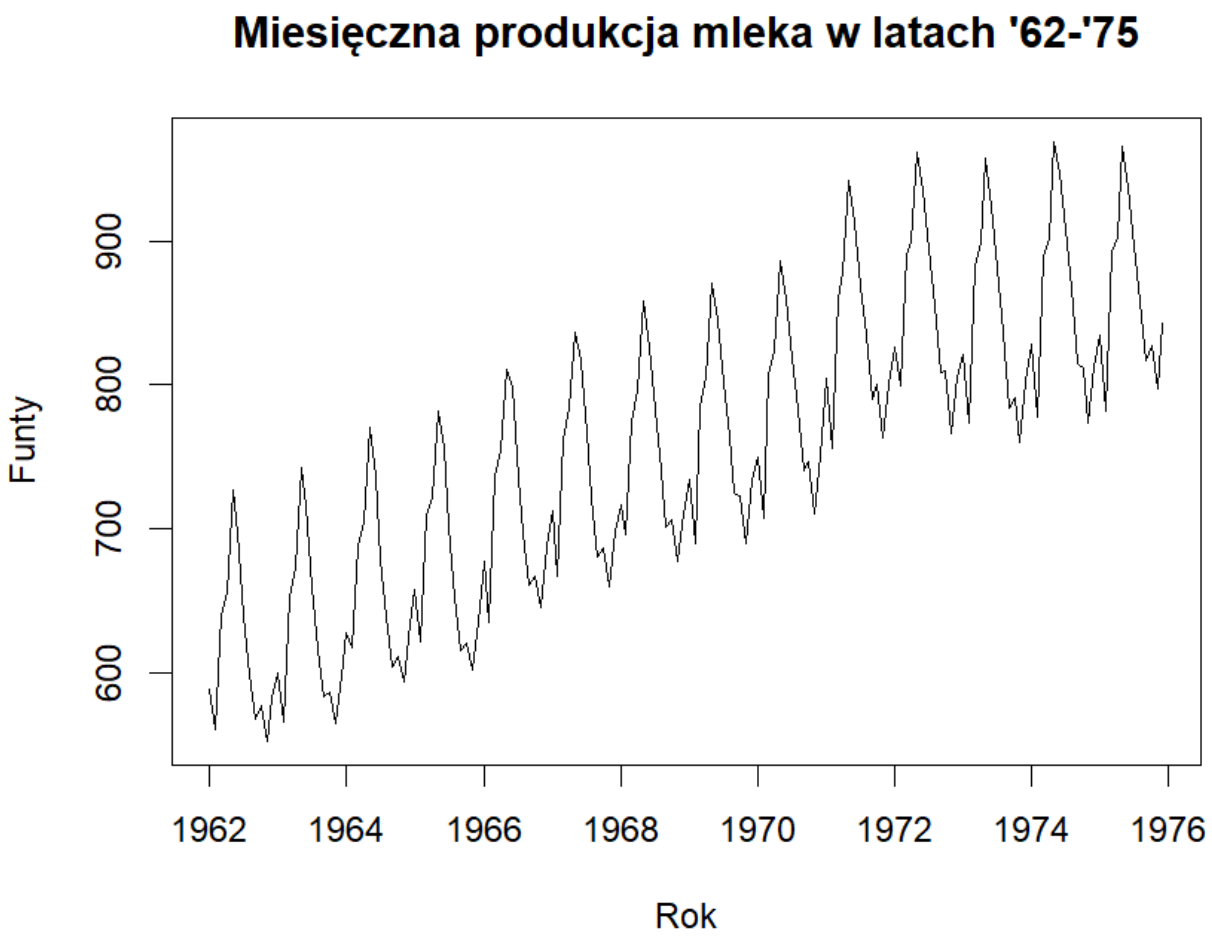
Za pomocą polecenia *summary* możemy wykonać wstępną analizę. Minimalna ilość wyprodukowanego mleka w podanym okresie to 553 funty, maksymalna 969 funty, a średnia miesięczna produkcja wynosiła około 754.7 funtów i jest to niewiele mniej od mediany, która wynosi 761 funtów.

```
> summary(milk_ts)
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max. 
  553.0   677.8   761.0   754.7   824.5   969.0 
```

2. Wykres szeregu czasowego, analiza trendu, wykresy sezonowe, analiza wahań sezonowych

Rysujemy wykres szeregu czasowego.

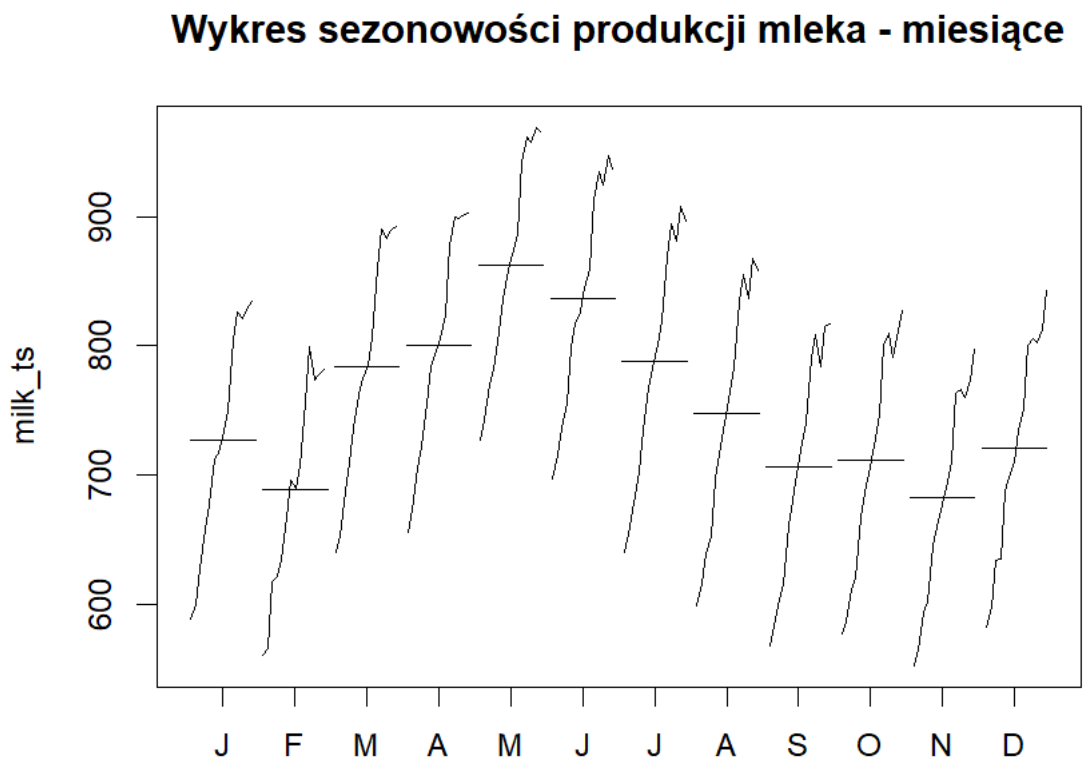
```
> plot(milk_ts, main = "Miesięczna produkcja mleka w latach '62-'75",
+      xlab = "Rok",ylab="Funty")
```



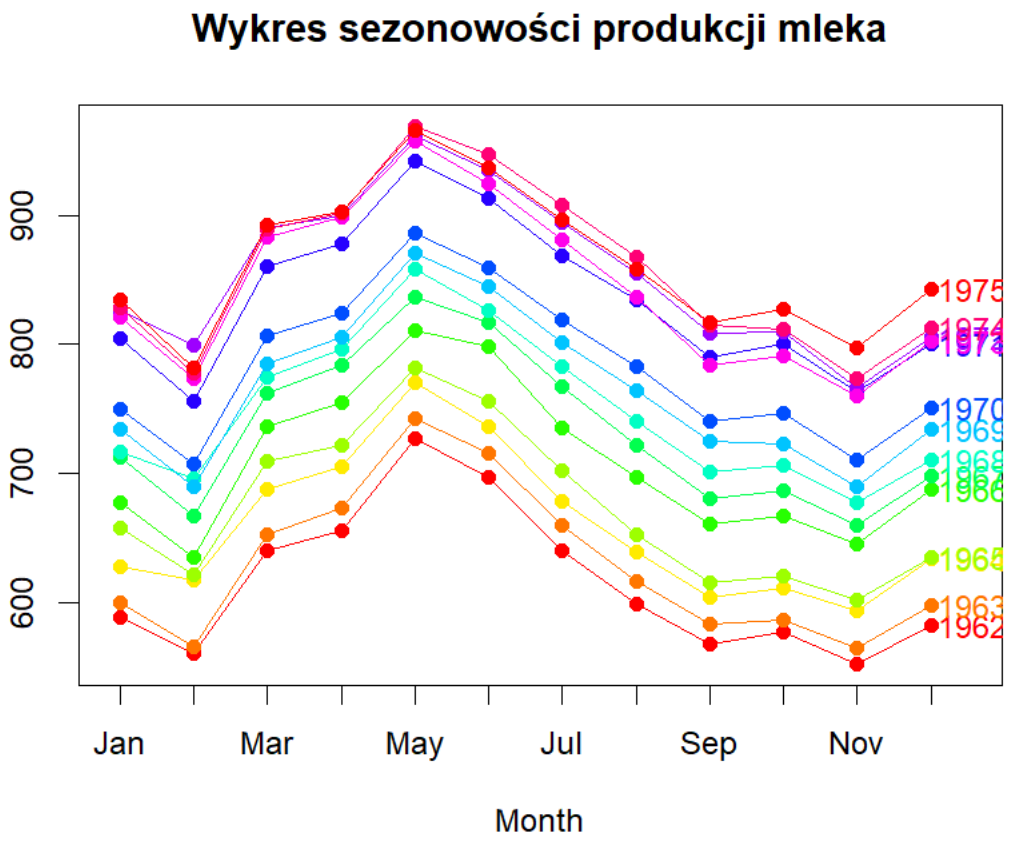
Wyraźnie widać sezonowość i trend wzrostowy. Największe wzrosty możemy zaobserwować w połowie roku, co mogą uzasadniać cieplejsze miesiące i większa ilość pożywienia w tym czasie. Amplituda wahań sezonowych pozostaje stabilna.

W celu wykonania wykresów sezonowych i ich analizy załadujemy pakiet *forecast*.

```
> install.packages("forecast")
> library(forecast)
> monthplot(milk_ts,main="Wykres sezonowości produkcji mleka")
> seasonplot(milk_ts,col=rainbow(13),
+            main="Wykres sezonowości produkcji mleka",
+            year.labels=TRUE,pch=19)
```



Wykres *monthplot* przedstawia średnie miesięczne wahania po usunięciu trendu. Patrząc na wykres łatwo stwierdzić istnienie sezonowości. Średnie wartości szeregu w miesiącach letnich jak wspomnieliśmy wcześniej są większe. Dodatkowo, w szeregach czasowych dla poszczególnych miesięcy możemy zauważyć trend wzrostowy, który jednak spada od maja i zmienia się w trend spadkowy.

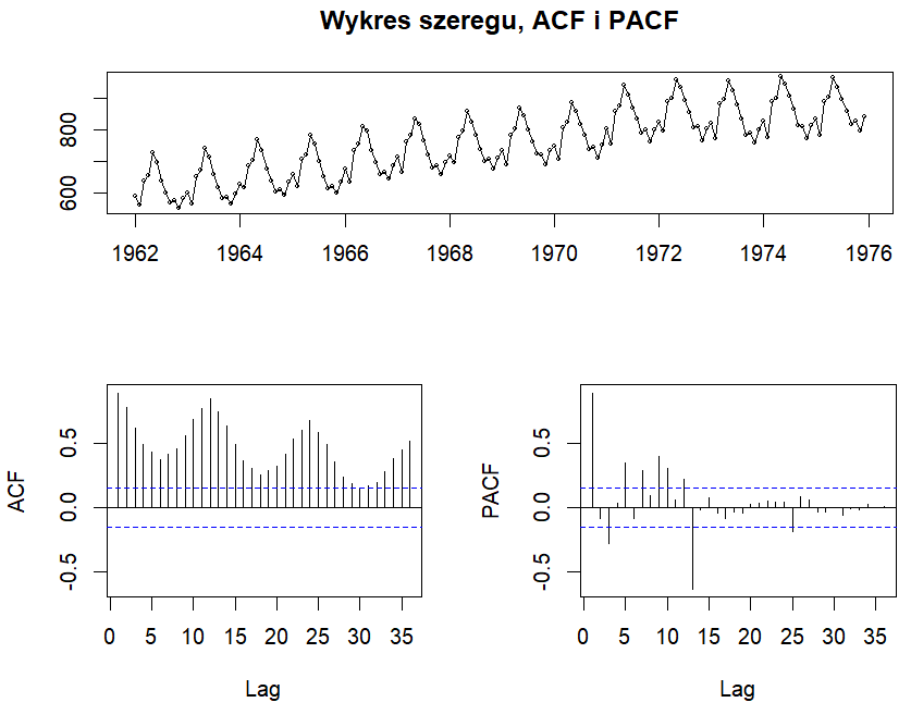


Wykres *seasonplot* przedstawia każdą serię roczną jako osobną linię, co pozwala porównać sezonowość między latami. Możemy zauważyć że ilość wyprodukowanego mleka stale wzrasta. Szczególną różnicę widać na przełomie lat 1970 i 1971 po czym następuje spowolnienie wzrostu.

3. Wykresy ACF, PACF

Rysujemy wykresy ACF i PACF dla szeregu czasowego. W tym celu użyjemy funkcji *tsdisplay.*

```
> tsdisplay(milk_ts,main="Wykres szeregu, ACF i PACF" )
```



Na wykresie autokorelacji wszystkie wartości wychodzą poza pas. Słupki mają sinusoidalny przebieg i powoli maleją. Wskazuję to na sezonowość roczną. Pierwsze wartości są bliskie jedynce co wskazuje na niestacjonarność danych i potrzebę ich różnicowania. Dla wykresu częściowej autokorelacji pierwsza wartość również jest bliska jeden tak jak kilka istotnych wartości w kolejnych latach. Sugeruję to obecność sezonowości.

4. Wykonanie niezbędnych przekształceń

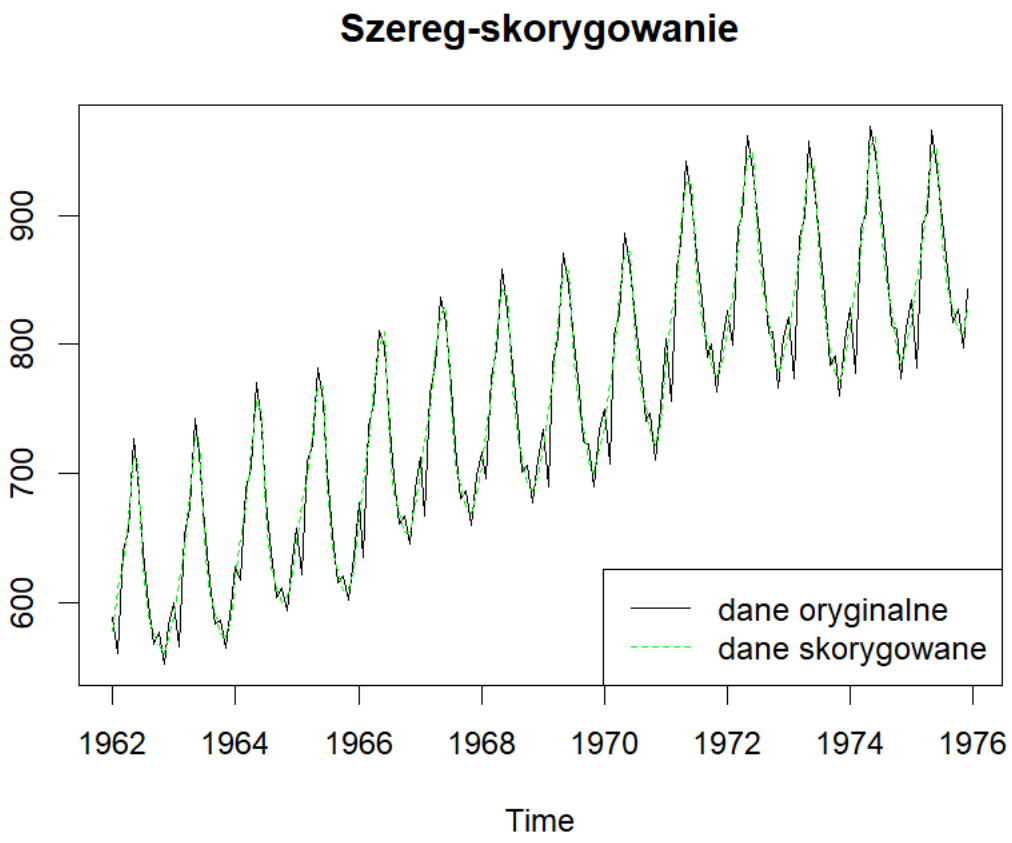
Korekta kalendarzowa

Pierwszym przekształceniem, które wykonamy będzie korekta kalendarzowa ze względu na różną liczbę dni w miesiącach.

$$dane\ skorygowane = \frac{dane}{liczba\ dni\ w\ miesiącu} \times \frac{365.25}{12}$$

Liczymy średnią liczbę dni w miesiącu. Funkcja *montdays()* zwraca miesięczną liczbę dni w . Po wykonaniu operacji mnożenia i dzielenia dostajemy skorygowane dane. Rysujemy wykres.

```
> srednia=365.25/12
> liczba=monthdays(milk_ts)
> milk.adj=milk_ts/liczba*srednia
> ts.plot(milk_ts,milk.adj,main="Szereg-skorygowanie",
+         col=c("black","green"),lty=c(1,2))
> legend("bottomright",legend=c("dane oryginalne", "dane skorygowane"),
+        col=c("black","green"),lty=c(1,2))
> x =milk.adj - milk_ts
```



Czarnym kolorem zaznaczono oryginalne dane, a na zielono dane skorygowane. Korekta różni się od danych oryginalnych. Szczególnie różnice występują pomiędzy sezonowymi skokami – większym i poprzedzającym go mniejszym.

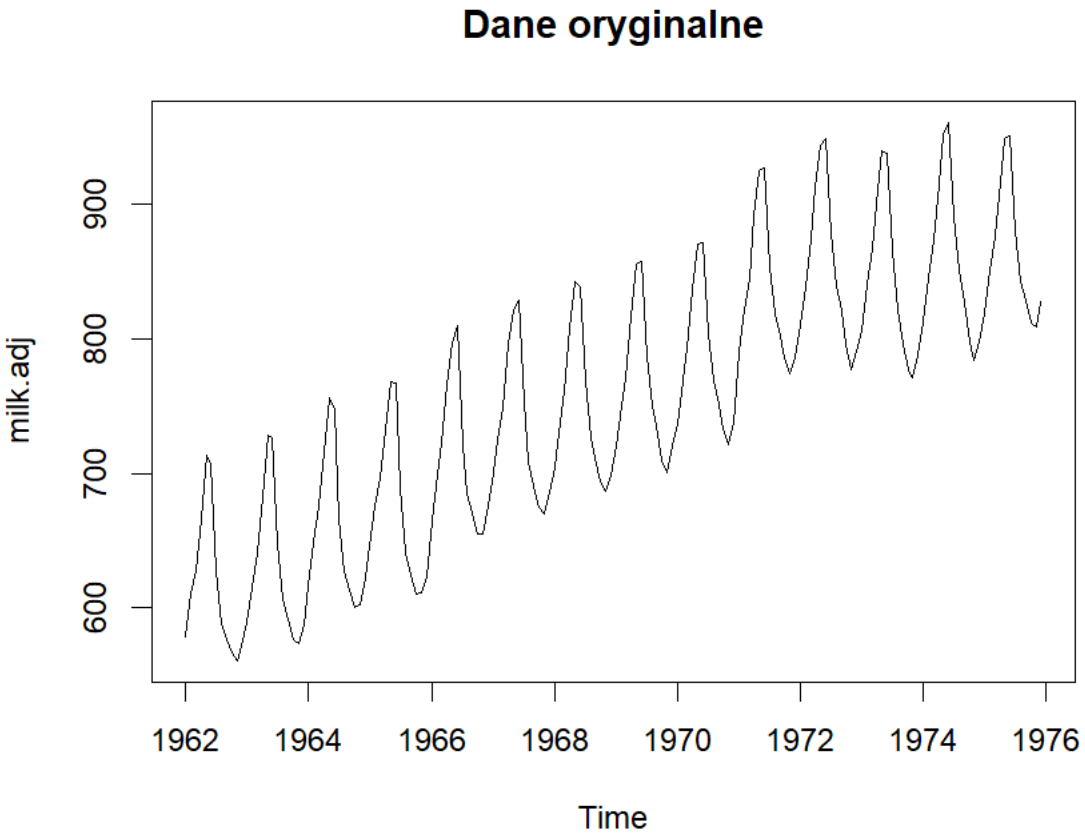
Przekształcenie Boxa-Coxa

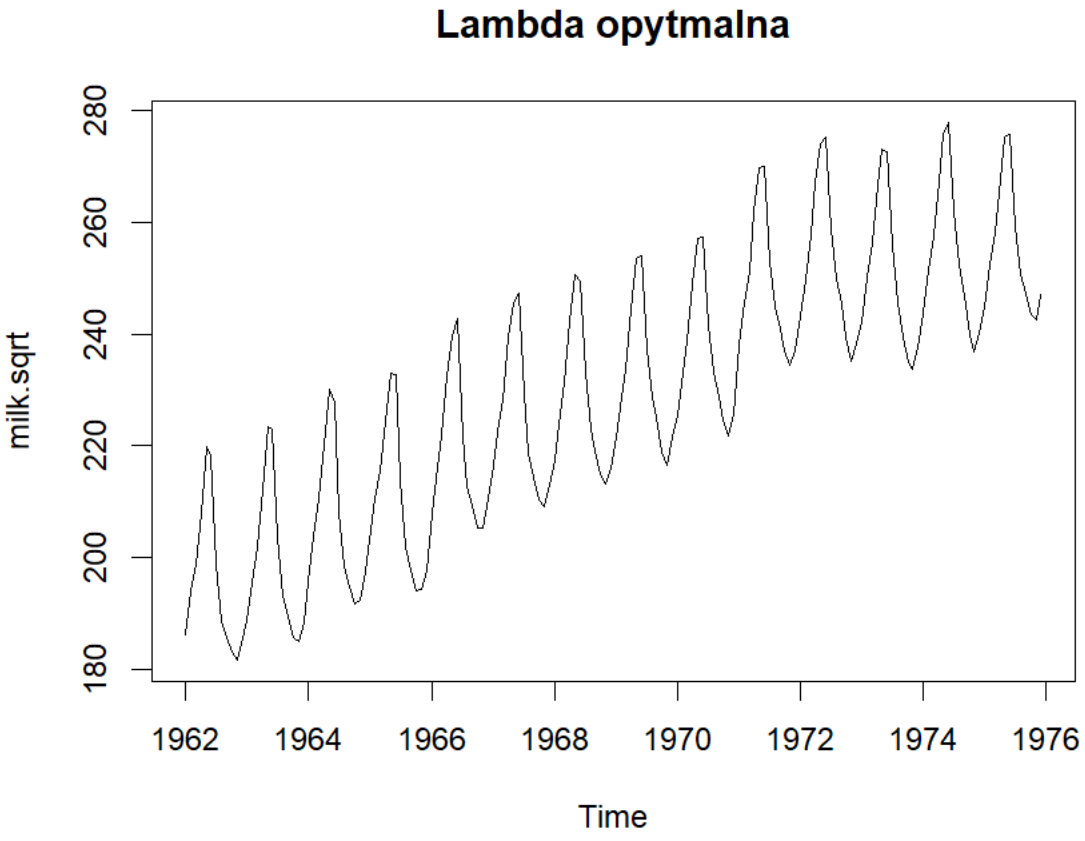
Wykonanie przekształceń Boxa- Coxa wydaję się zbędne ze względu na brak wzrostów lub spadków amplitudy wahań sezonowych w kolejnych okresach i braku dużego wpływu obserwacji odstających. W celu zweryfikowania tej tezy wykonamy przekształcenie dla optymalnej lambdy, którą otrzymamy z funkcji *BoxCox.lambda*.

```
> BoxCox.lambda(milk_ts)
[1] 0.7847705
```

Przekształcamy i rysujemy wykresy.

```
> milk.sqrt=BoxCox(milk.adj,lambda= 0.7847705)
> milk.log=BoxCox(milk.adj,lambda=0)
> plot(milk.adj,main="dane oryginalne")
> plot(milk.sqrt,main="lambda opytmalna")
```



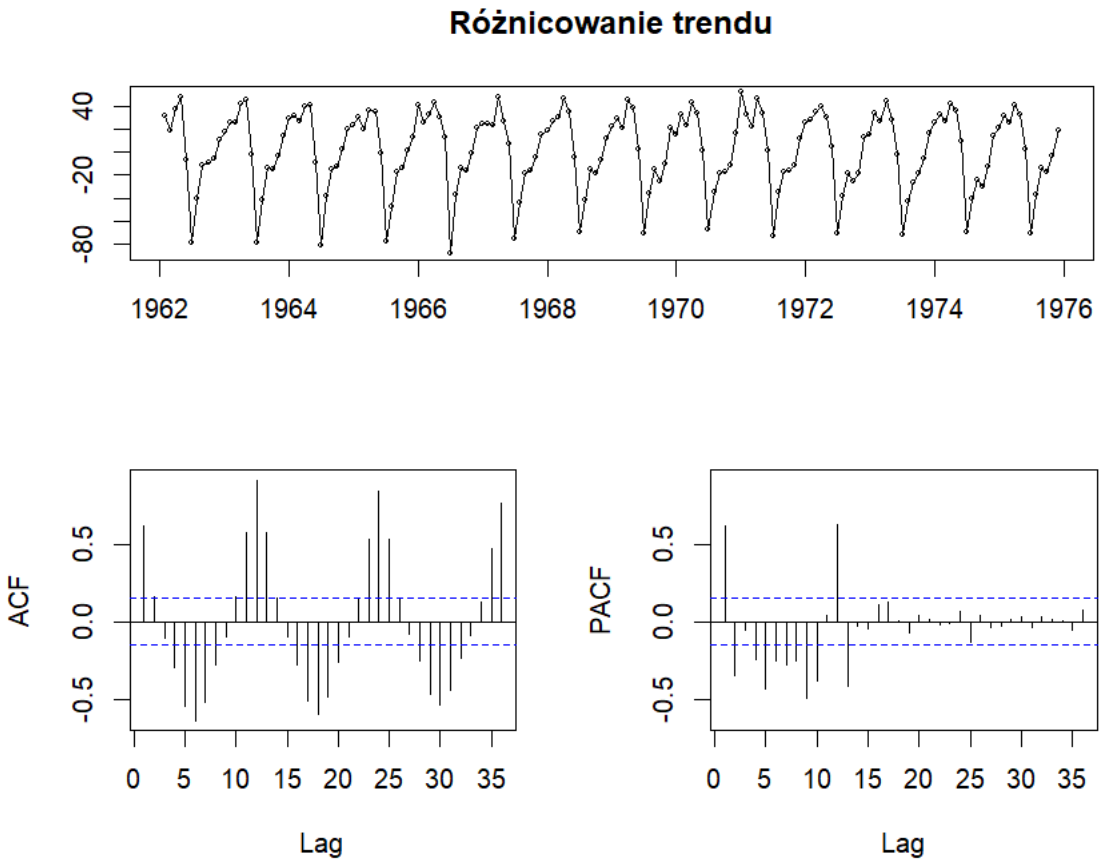


Rzeczywiście różnice są nie zauważalne.

Różnicowanie

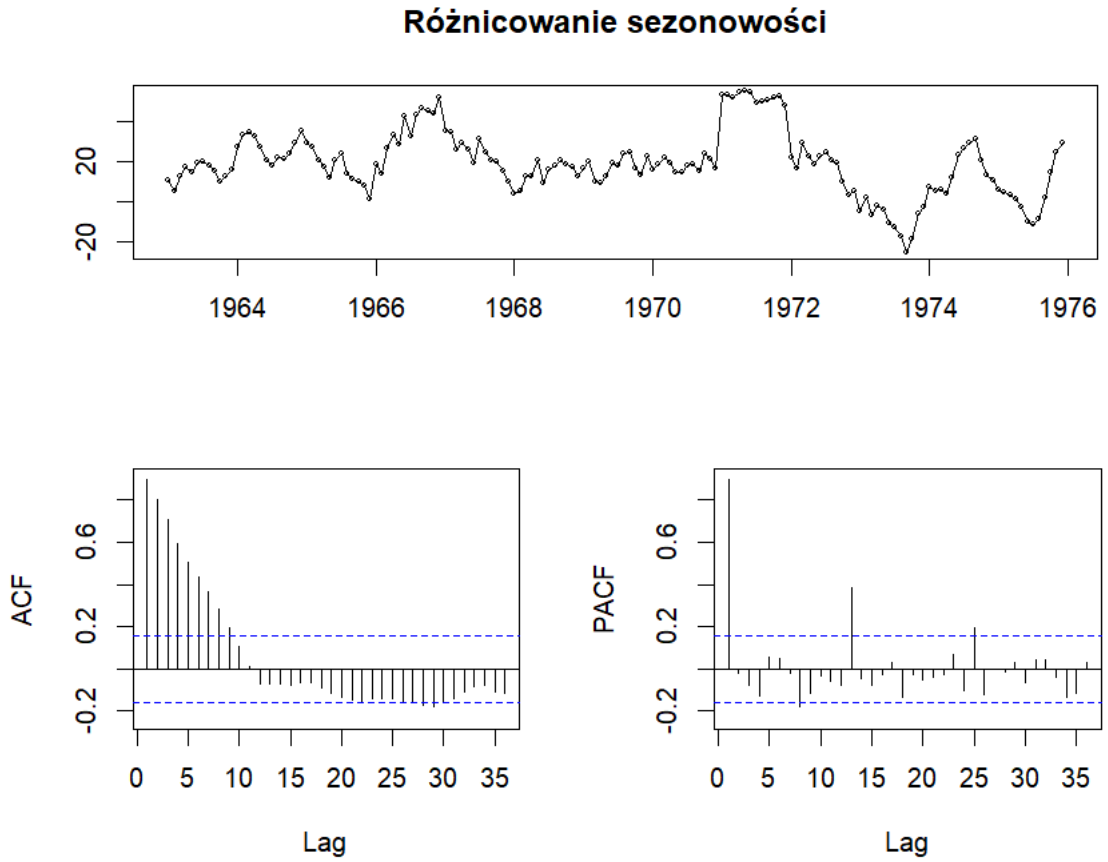
Wykonamy operację różnicowania dla skorygowanego szeregu czasowego w celu wyeliminowania trendu oraz współrzędnej sezonowej. Różnicowanie zrobimy za pomocą komendy *diff*.

```
> milk.diff.trend=diff(milk_ts,differences=1)
> tsdisplay(milk.diff.trend)
```



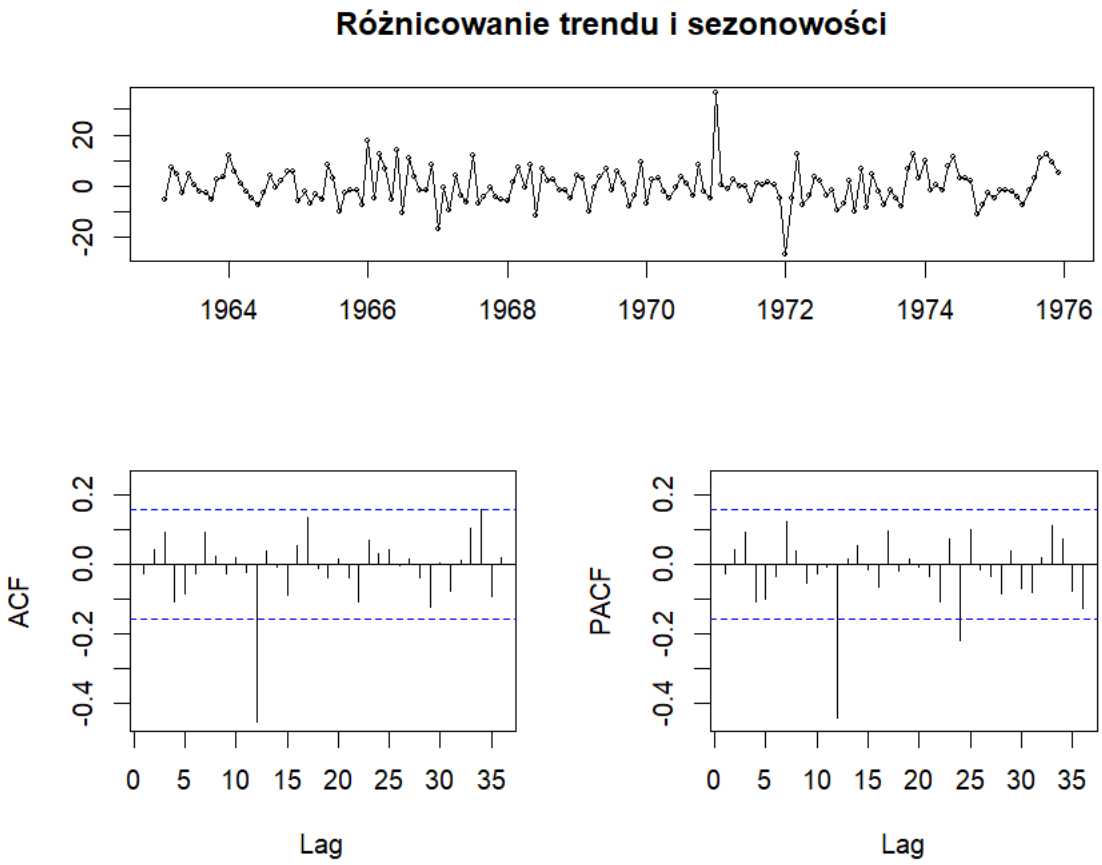
Po różnicowaniu szeregu możemy stwierdzić brak trendu, ale nadal widać wyraźną sezonowość. Wykres ACF może to potwierdzić z nadal widocznym sinuidalnym kształtem. Zróżnicujujmy teraz sezonowość.

```
> tsdisplay(milk.diff.sez,main="Różnicowanie sezonowości")
> milk.diff=diff(milk_ts,differences=1, lag=12)
```



Szereg nie wykazuje cech sezonowości. Na wykresie PACF cztery słupki wychodzą poza linie, a reszta jest nie istotna. Możemy stwierdzić, że wykonanie różnicowania sezonowości zdecydowanie pomogło zbliżyć się danym do kształtu białego szumu. Spróbujmy zastosować różnicowanie trendu i sezonowości razem.

```
> milk.diff=diff(diff(milk.adj, lag=12),differences=1)
> tsdisplay(milk.diff, main="Różnicowanie trendu i sezonowości")
```



Metoda ta dała nam najlepszy rezultat. Szereg jest zbliżony do stacjonarnego. Wykresy ACF i PACF nie wskazują na obecność wyraźnych wzorców. Dla obu wykresów ACF i PACF tylko dwa słupki są istotne.

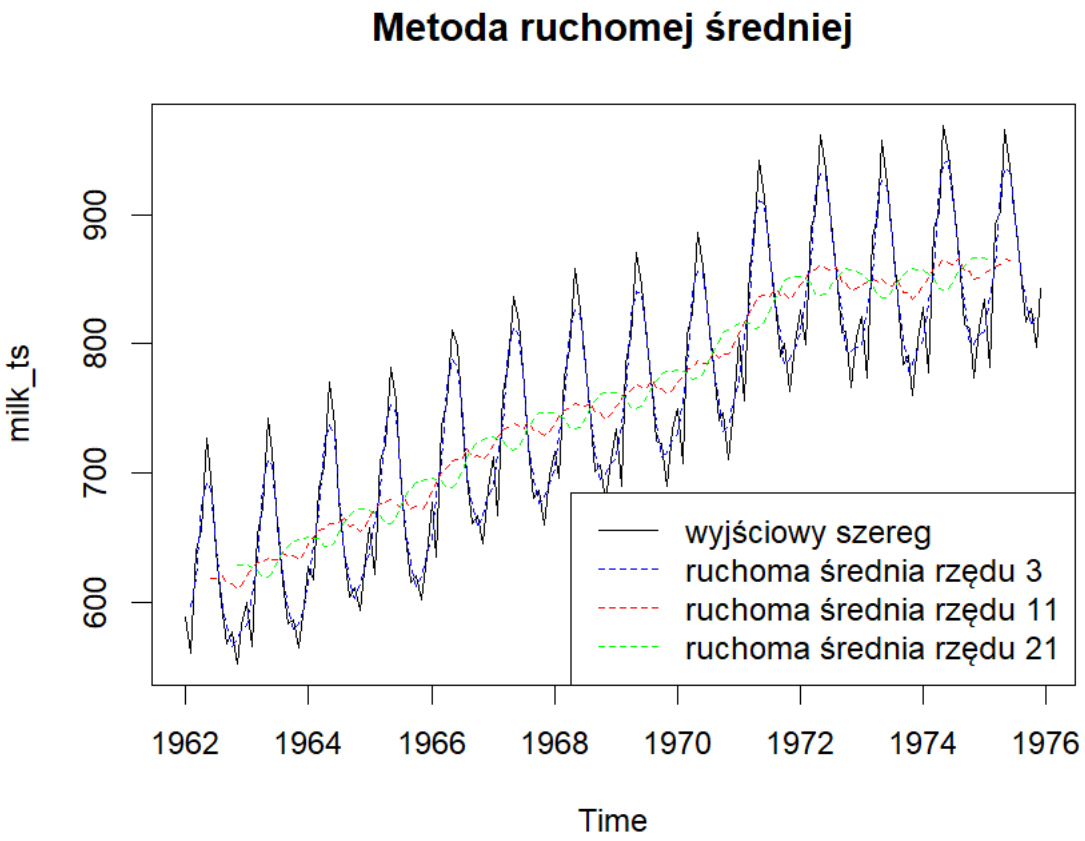
5. Wygładzanie szeregu czasowego

Do wygładzenia naszego szeregu wykorzystamy trzy sposoby: średnią ruchomą, ważoną średnią ruchomą i wygładzenie wykładnicze.

Średnia ruchoma

W celu wygładzenia szeregu czasowego średnią ruchomą wykorzystamy funkcję *filter* rzędu 3, 11 i 21.

```
> m21=filter(milk_ts,filter=rep(1/21,21),sides=2)
> m3=filter(milk_ts,filter=rep(1/3,3),sides=2)
> m11=filter(milk_ts,filter=rep(1/11,11),sides=2)
> plot(milk_ts,main="Metoda ruchomej średniej")
> lines(m3,col="blue",lty=2)
> lines(m11,col="red",lty=2)
> lines(m21,col="green",lty=2)
> legend("bottomright",legend =c("wyjściowy szereg", "ruchoma średnia rzędu 3",
"ruchoma średnia rzędu 11","ruchoma średnia rzędu 21"), col=c("black","blue","re
d",  "green"),lty=c(1,2,2,2))
```

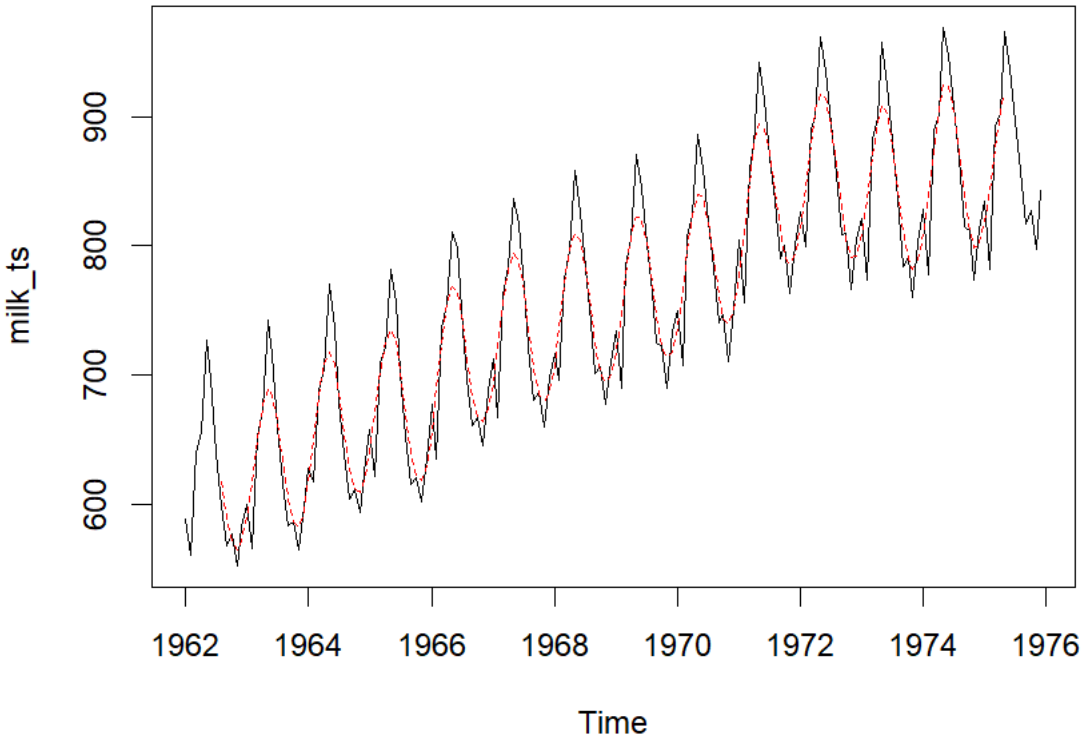


Na niebiesko zostało zaznaczone przekształcenie rzędu 3, na czerwono rzędu 11, a na zielono rzędu 21. Na czarno prezentowane są dane oryginalne. Możemy zauważyć, że im wyższy rząd tym większe wygładzenie. Najlepszym wygładzeniem wydaję się być wygładzenie rzędu 3, ponieważ oddaje kształt danych i wystarczająco wygładza.

Ważona średnia ruchoma

Sprawdźmy teraz ruchomą średnią ważoną. Do jej stworzenia wykorzystamy filtr Spensera z wagami {-3, -6, -5, 3, 21, 46, 67, 74, 67, 46, 21, 3, -5, -6, -3} i z filtrem symetrycznym.

```
> spenser=filter(milk_ts,filter=(1/320)*
+                  c(-3,-6,-5,3,21,46,67,74,67,46,21,3,-5,-6,-3),sides=2)
> plot(milk_ts)
> lines(spenser,col="red",lty=2)
```

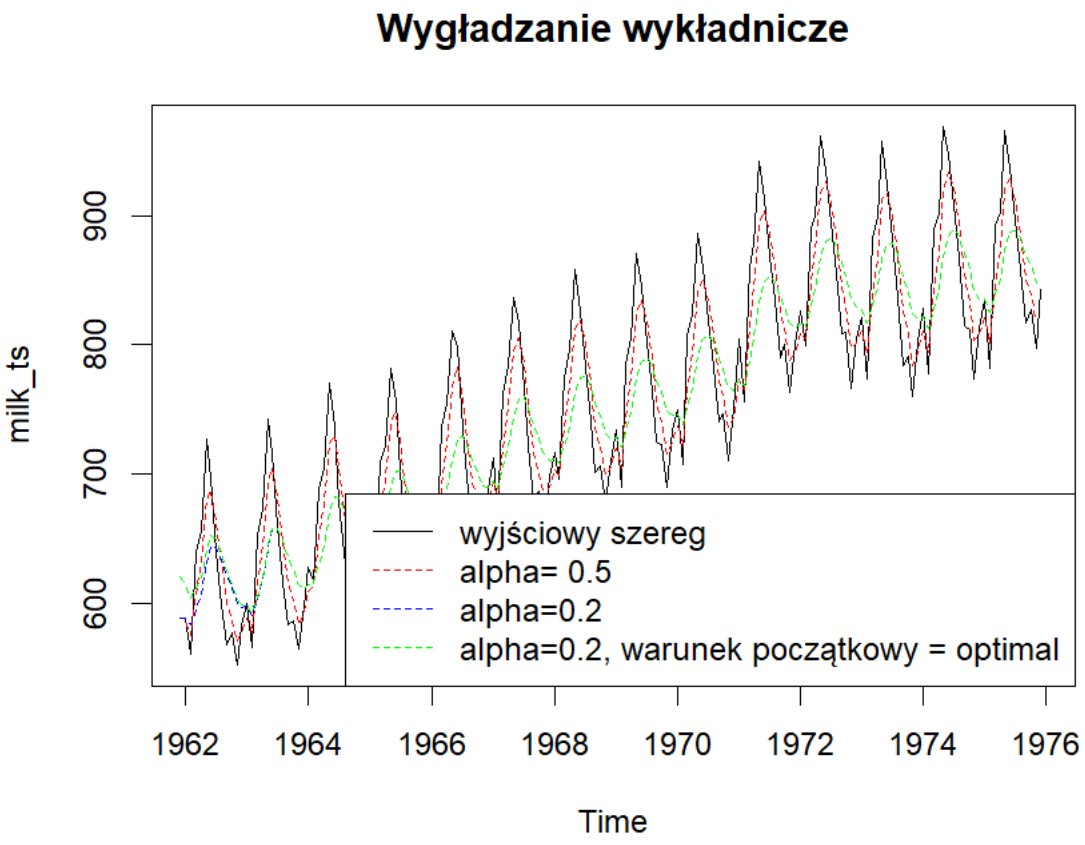


Wygładzenie jest podobne do wygładzenia średnią ruchomą rzędu 3. Trend i sezonowość nie zostały zniekształcone.

Wygładzenie wykładnicze

Ostatnią metodą jest wygładzanie wykładnicze. Wybierzmy równe kolejno 0,5 i 0,2 oraz dla α =0,2 wybierzmy warunek początkowy i optymalny.

```
> milk.1=ses(milk_ts,alpha=0.5,initial="simple")
> milk.2=ses(milk_ts,alpha=0.2,initial="simple")
> milk.3=ses(milk_ts,alpha=0.2,initial="optimal")
> milk.close.1=lag(fitted(milk.1))
> milk.close.2=lag(fitted(milk.2))
> milk.close.3=lag(fitted(milk.3))
> plot(milk_ts,main="Wygładzanie wykładnicze")
> lines(milk.close.1,col="red",lty=2)
> lines(milk.close.2,col="blue",lty=2)
> lines(milk.close.3,col="green", lty=2)
> legend("bottomright",legend= c("wyjściowy szereg","alpha= 0.5","alpha=0.2",
+                                "alpha=0.2, warunek początkowy = optimal"),
+        col=c("black","red","blue", "green"),lty=c(1,2,2,2))
```



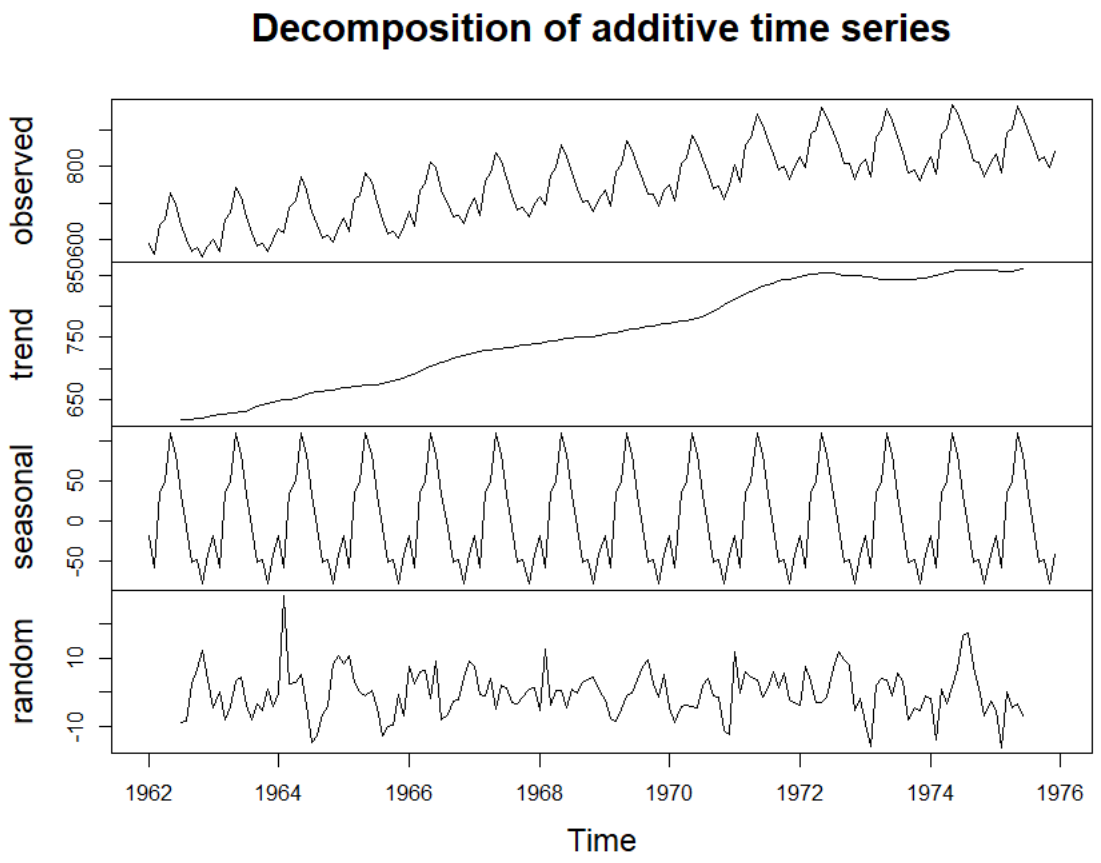
Na czerwono oznaczono wygładzenie z parametrem α=0,5, na niebiesko α=0,2 z warunkiem początkowym, a na zielono α=0,2 z optymalnym warunkiem początkowym. Dane oryginalne są zaprezentowane na czarno. Pierwszą rzeczą, która rzuca się w oczy to nałożenie się wykresów dla =0,2 w obu przypadkach. Możemy stwierdzić, że warunek początkowy jest warunkiem optymalnym. Wygładzenie z parametrem =0,5 jest najbliższe oryginalnym danym. Z tego względu uznajemy, że parametr α=0,2 daje lepsze wygładzenie.
Trudno zdecydować, która metoda dała najlepsze wygładzenie ze względu na podobieństwo rezultatów. Możemy uznać, że wszystkie przedstawione metody dały zadowalający efekt.

6. Dekompozycja szeregu czasowego

Dekompozycja addytywna

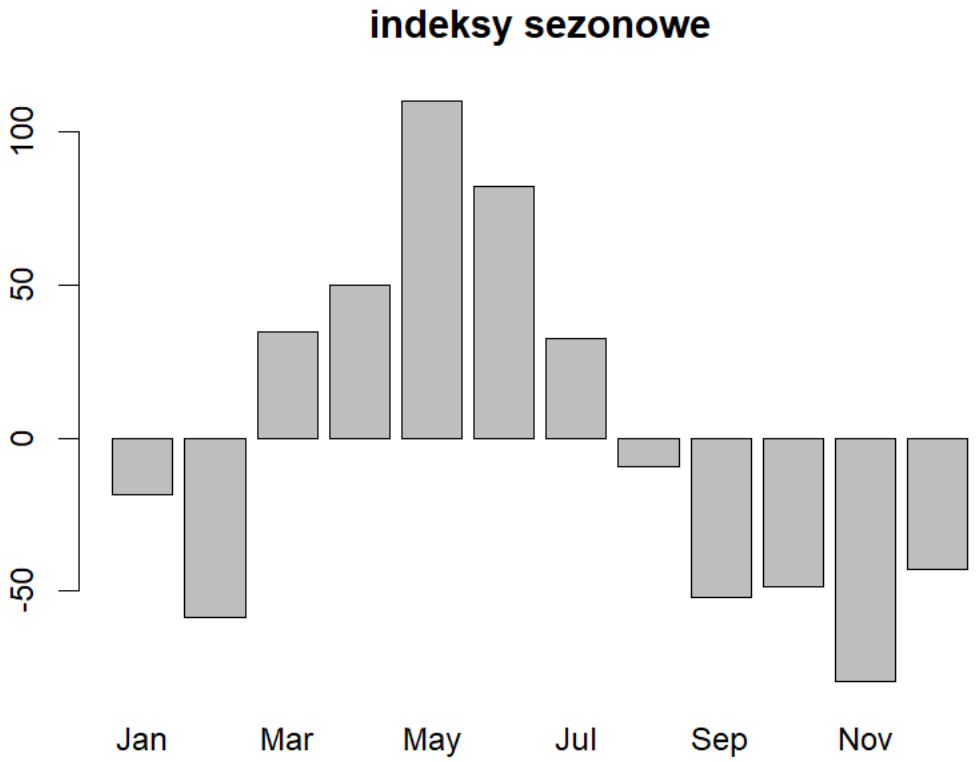
Do danych *milk_ts* zastosujemy dekompozycje addytywną, narysujemy wykres i podzielimy na poszczególne składniki.

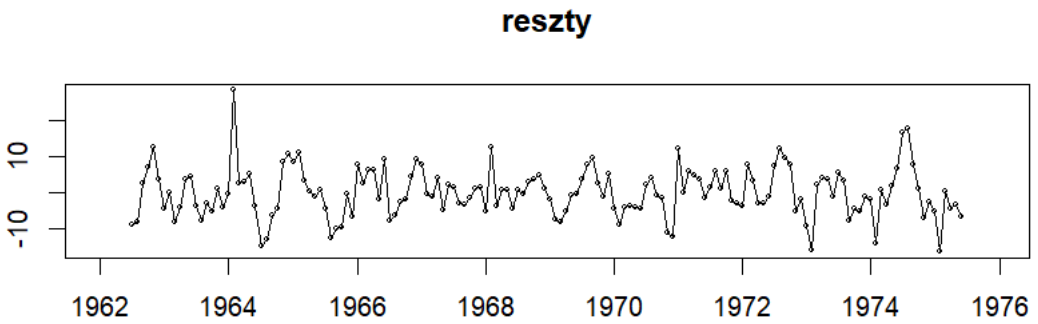
```
> milk.ad=decompose(milk_ts,type="additive")
> plot(milk.ad)
> milk.ad.trend=milk.ad$trend
> milk.ad.sez=milk.ad$seasonal
> milk.ad.ind.sez=milk.ad$figure
> milk.ad.reszty=milk.ad$random
```

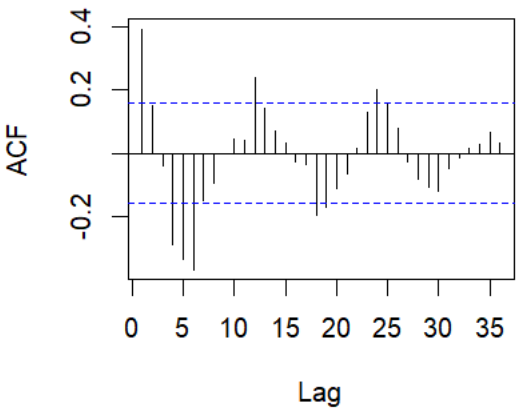


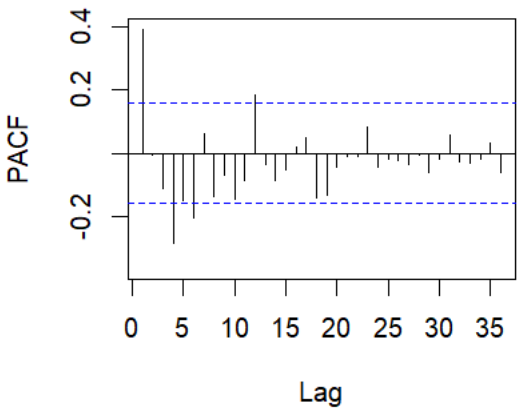
Stwórzmy również wykres słupkowy i wywołajmy funkcję *tsdisplay*.

```
> barplot(milk.ad.ind.sez,names.arg=month.abb, main="indeksy sezonowe")
> tsdisplay(milk.ad.reszty,main="reszty")
```


indeksy sezonowe
100
50
0
-50
Jan
Mar
May
Jul
Sep
Nov


reszty
10
-10
1962
1964
1966
1968
1970
1972
1974
1976


ACF
0.4
0.2
-0.2
0
5
10
15
20
25
30
35
Lag

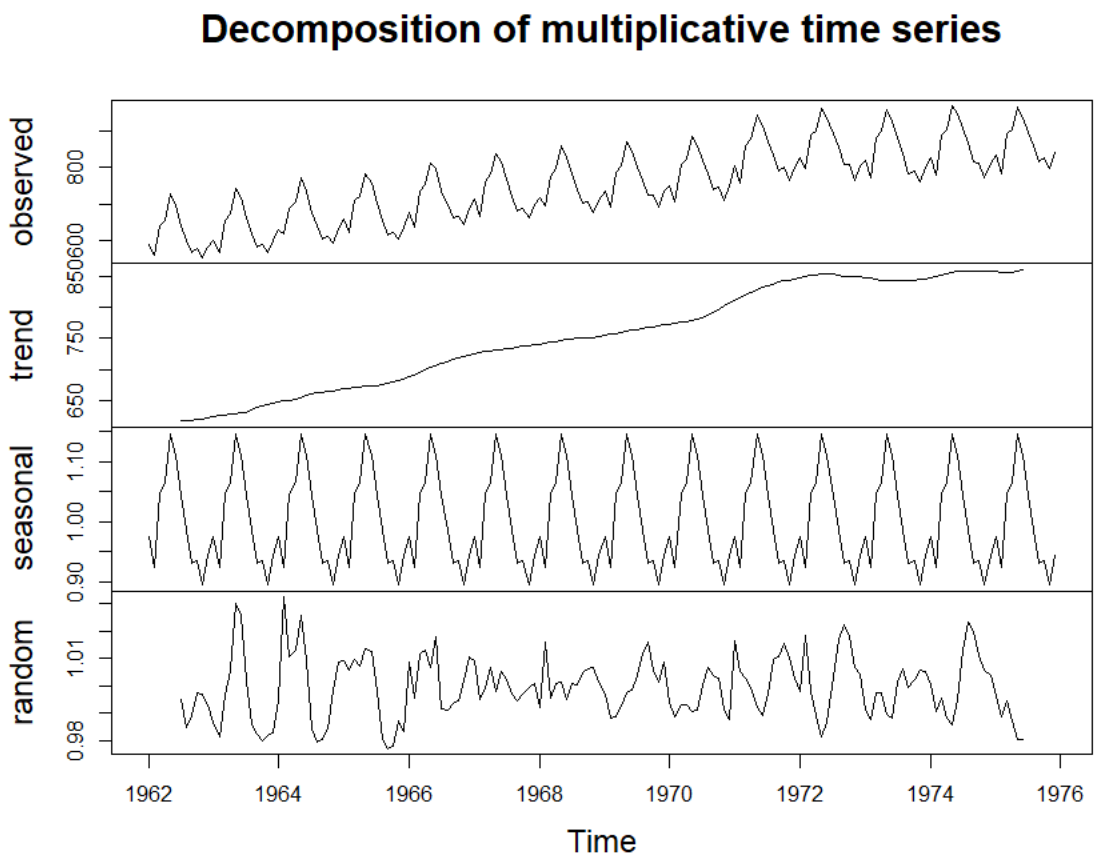

PACF
0.4
0.2
-0.2
0
5
10
15
20
25
30
35
Lag

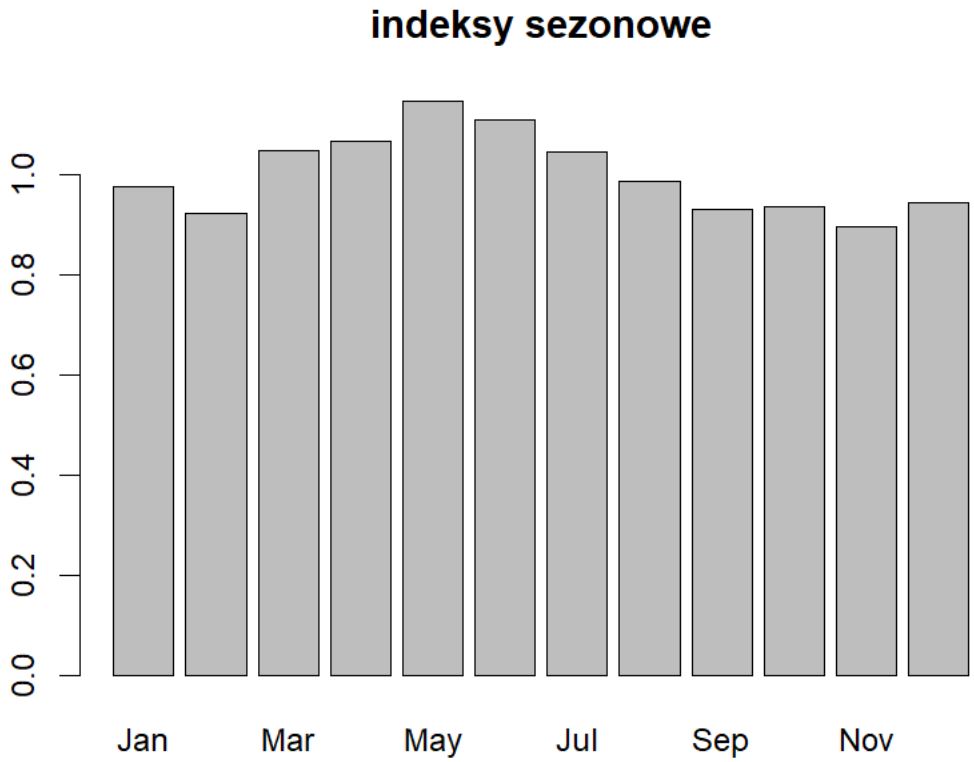
Wykres reszt wygląda w pewnym stopniu na stacjonarny, ale nie jest to najlepsze przekształcenie zważając na wykresy ACF, który przybiera sinusoidalny kształt i PACF gdzie cztery słupki są istotne, co wskazuje na autokorelacje. Jest to przekształcenie akceptowalne.

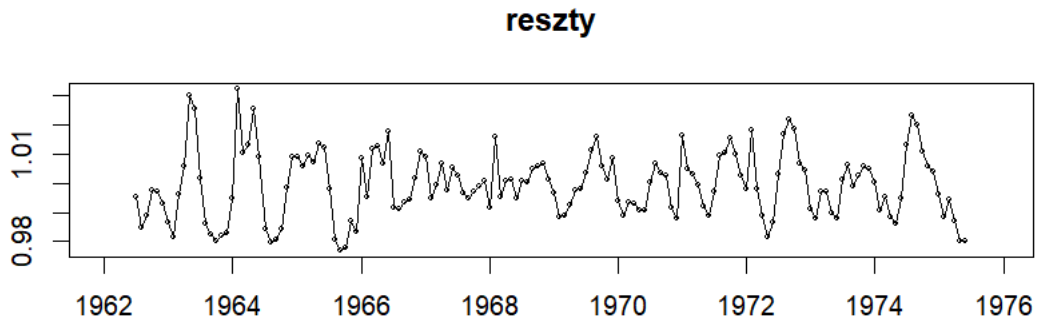
Dekompozycja multiplikatywna

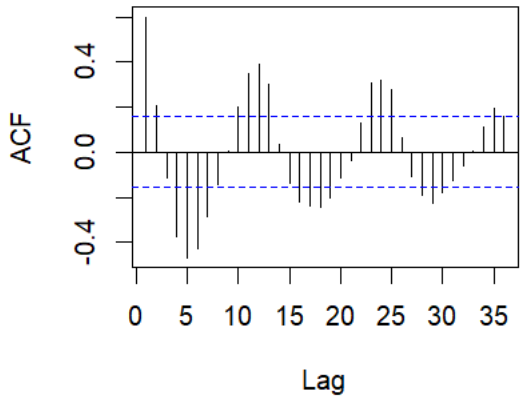
Sprawdźmy dekompozycję multiplikatywną. Wykonamy te same kroki co w dekompozycji addytywnej zmieniając typ na multiplikatywny.

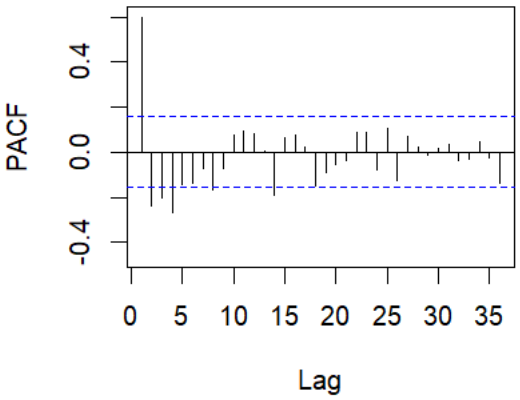
```
> milk.mul=decompose(milk_ts,type="multiplicative")
> plot(milk.mul)
> milk.mul.trend=milk.mul$trend
> milk.mul.sez=milk.mul$seasonal
> milk.mul.ind.sez=milk.mul$figure
> milk.mul.reszty=milk.mul$random
> barplot(milk.mul.ind.sez,names.arg=month.abb, main="indeksy sezonowe")
> tsdisplay(milk.mul.reszty,main="reszty")
```




indeksy sezonowe
0.0
0.2
0.4
0.6
0.8
1.0
Jan
Mar
May
Jul
Sep
Nov


reszty
0.98
1.01
1962
1964
1966
1968
1970
1972
1974
1976


ACF
-0.4
0.0
0.4
0
5
10
15
20
25
30
35
Lag


PACF
-0.4
0.0
0.4
0
5
10
15
20
25
30
35
Lag

Obie metody dają bardzo podobne wyniki. Możemy zauważyć sezonowość zbioru. Wykres ACF jest kształtem zbliżony do wykresu ACF w dekompozycji addytywnej. Na wykresie PACF możemy zauważyć więcej istotnych słupków. Jak we wcześniejszym przypadku przekształcenie jest akceptowalne.

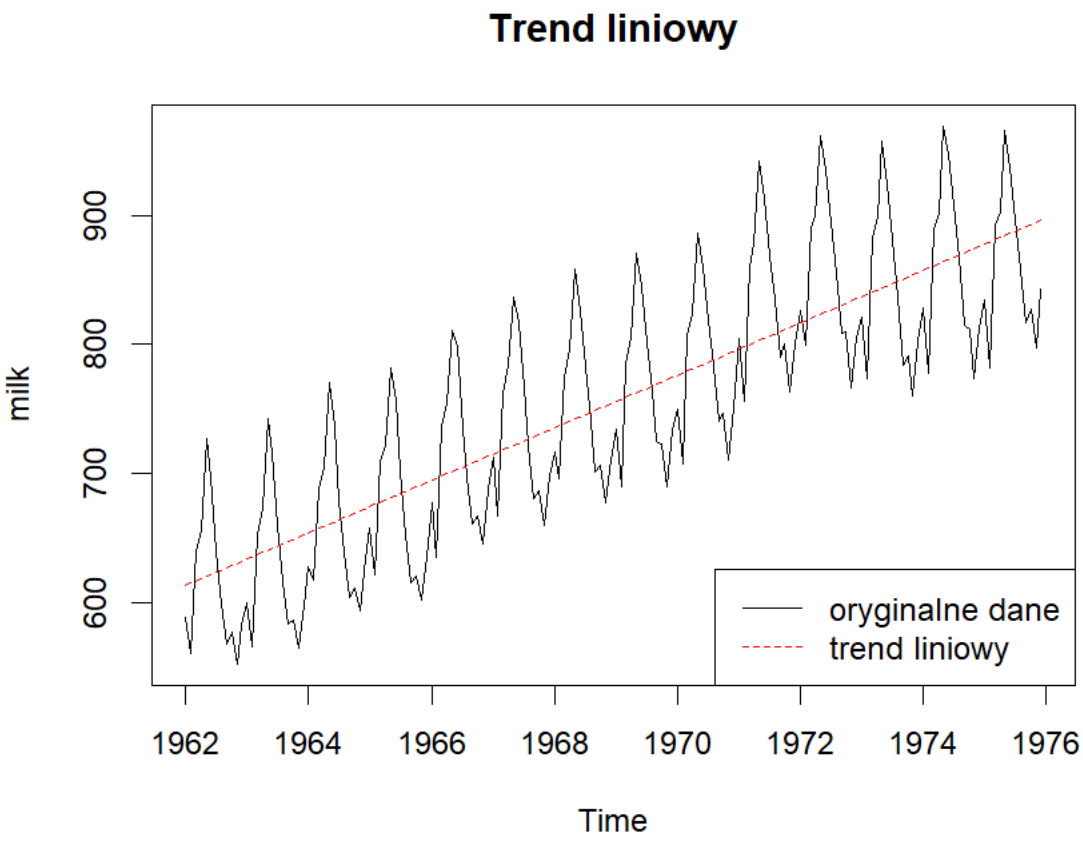
Trend liniowy

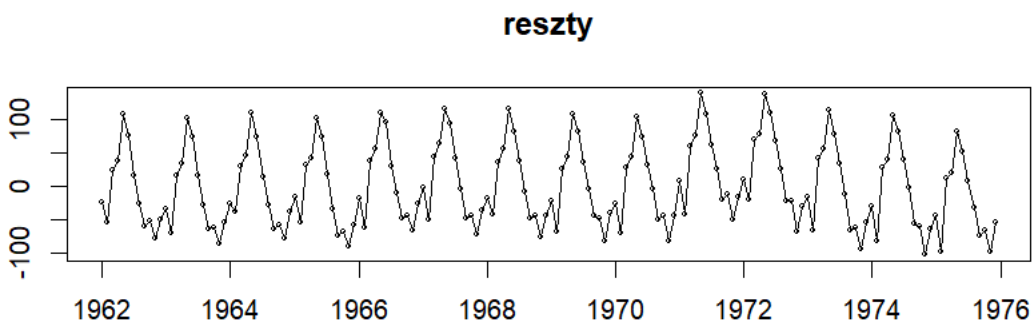
Następną metodą jest trend liniowy. Do jej stworzenia wykorzystamy funkcję *tslm* z pakietu *forecast*.

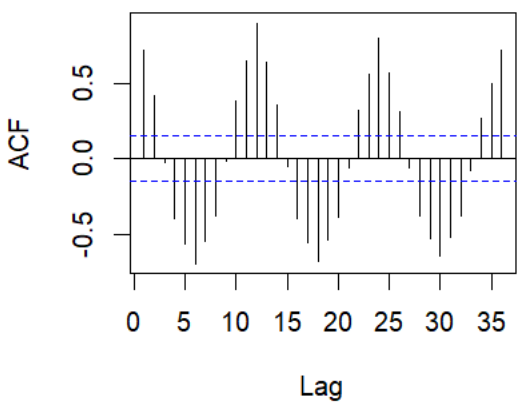
```
> milk.tlsm.trend= tslm(milk~trend)
> summary(milk.tlsm.trend)

Call:
tslm(formula = milk ~ trend)

Residuals:
    Min      1Q  Median      3Q     Max 
-101.04  -50.02  -15.30   42.88  139.05 

Coefficients:
             Estimate Std. Error t value Pr(>|t|)    
(Intercept) 611.68235    9.41444   64.97   <2e-16 ***
trend         1.69262    0.09663   17.52   <2e-16 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 60.74 on 166 degrees of freedom
Multiple R-squared:  0.6489,	Adjusted R-squared:  0.6468 
F-statistic: 306.8 on 1 and 166 DF,  p-value: < 2.2e-16
```

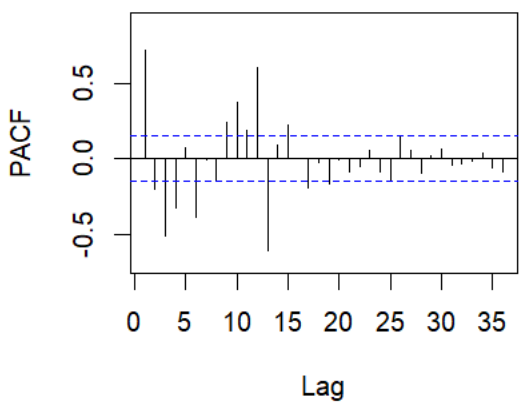
Estymator trendu liniowego wynosi: m_t = 611,68 + 1,69t. współczynniki są istotne statystycznie; R^2=0,65.

```
> plot(milk,main="Trend liniowy")
> lines(fitted(milk.tlsm.trend), col="red",lty=2)
> legend("bottomright",legend =c("oryginalne dane","trend liniowy"),
+        col=c("black", "red"),lty=c(1,2))
> tsdisplay(residuals(milk.tlsm.trend),main="reszty")
```


Trend liniowy
milk
600
700
800
900
1962
1964
1966
1968
1970
1972
1974
1976
Time
oryginalne dane
trend liniowy


reszty
-100
0
100
1962
1964
1966
1968
1970
1972
1974
1976


ACF
-0.5
0.0
0.5
0
5
10
15
20
25
30
35
Lag

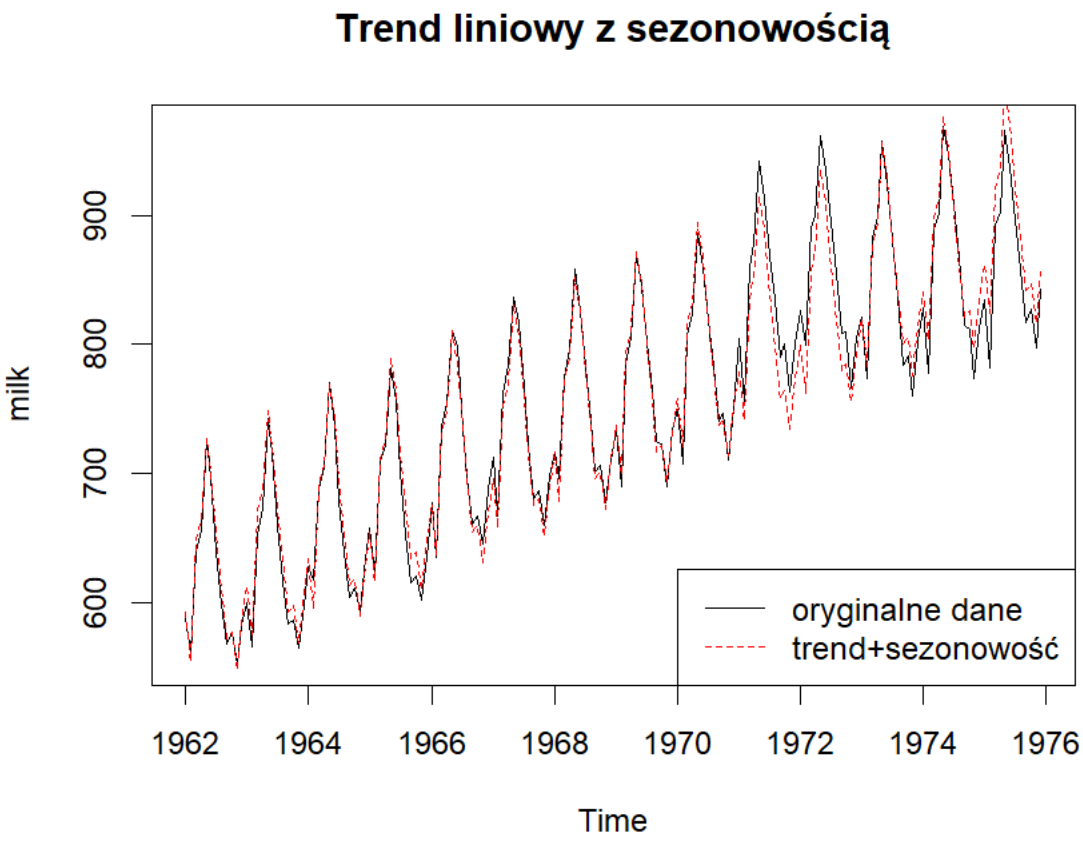

PACF
-0.5
0.0
0.5
0
5
10
15
20
25
30
35
Lag

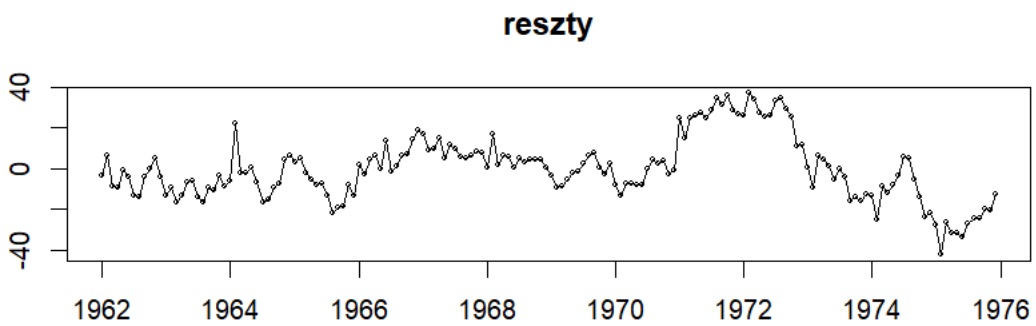
Współczynnik trendu jest istotny i czerwona linia dobrze oddaje tępo wzrostu wartości szeregu. Na wykresie reszt widoczna jest sezonowość, podobnie na wykresach ACF i PACF. Spróbujmy dodać współczynnik sezonowości.

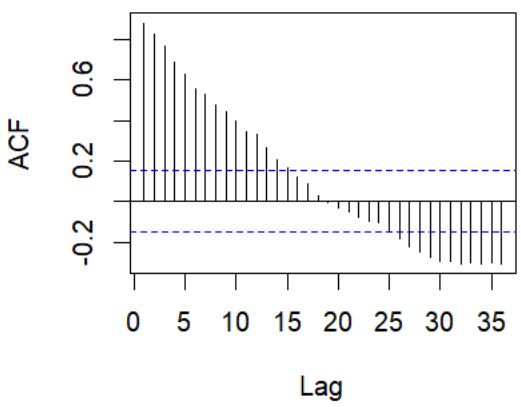
```
> milk.tlsm.trend.sez=tslm(milk~trend+season)
> summary(milk.tlsm.trend.sez)

Call:
tslm(formula = milk ~ trend + season)

Residuals:
    Min      1Q  Median      3Q     Max 
-42.051  -9.315  -1.060   6.296  37.148 

Coefficients:
             Estimate Std. Error t value Pr(>|t|)    
(Intercept) 590.57819    4.76977 123.817  < 2e-16 ***
trend         1.72776    0.02575  67.107  < 2e-16 ***
season2     -39.51348    6.10148  -6.476 1.18e-09 ***
season3      52.97305    6.10164   8.682 5.07e-15 ***
season4      67.81671    6.10191  11.114  < 2e-16 ***
season5     128.80324    6.10229  21.107  < 2e-16 ***
season6     100.43262    6.10278  16.457  < 2e-16 ***
season7      50.63342    6.10338   8.296 4.89e-14 ***
season8       8.33423    6.10408   1.365    0.174    
season9     -34.25067    6.10490  -5.610 9.06e-08 ***
season10    -30.76415    6.10582  -5.038 1.29e-06 ***
season11    -61.77763    6.10685 -10.116  < 2e-16 ***
season12    -25.07682    6.10799  -4.106 6.51e-05 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 16.14 on 155 degrees of freedom
Multiple R-squared:  0.9768,	Adjusted R-squared:  0.9751 
F-statistic: 544.9 on 12 and 155 DF,  p-value: < 2.2e-16
```

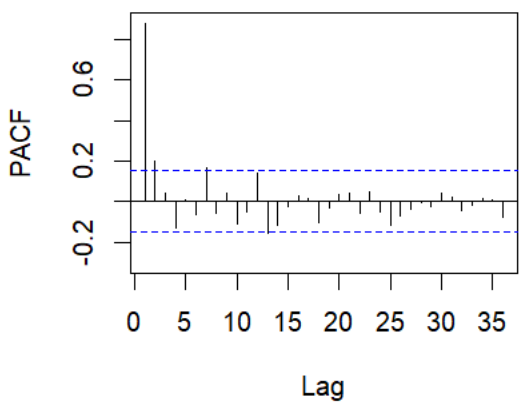
Estymator trendu liniowego wynosi: m_t = 590,58 + 1,73t. Prawie wszystkie współczynniki (oprócz season8) są istotne statystycznie; R^2=0,98. Popatrzmy na wykresy.

```
> plot(milk,main="Trend liniowy z sezonowością")
> lines(fitted(milk.tlsm.trend.sez),col="red",lty=2)
> legend("bottomright",legend=c("oryginalne dane", "trend+sezonowość"),
+        col=c("black","red"),lty=c(1,2))
> tsdisplay(residuals(milk.tlsm.trend.sez),main="reszty")
```


Trend liniowy z sezonowością
milk
600
700
800
900
1962
1964
1966
1968
1970
1972
1974
1976
Time
oryginalne dane
trend+sezonowość


reszty
40
0
-40
1962
1964
1966
1968
1970
1972
1974
1976


ACF
0.6
0.2
-0.2
0
5
10
15
20
25
30
35
Lag

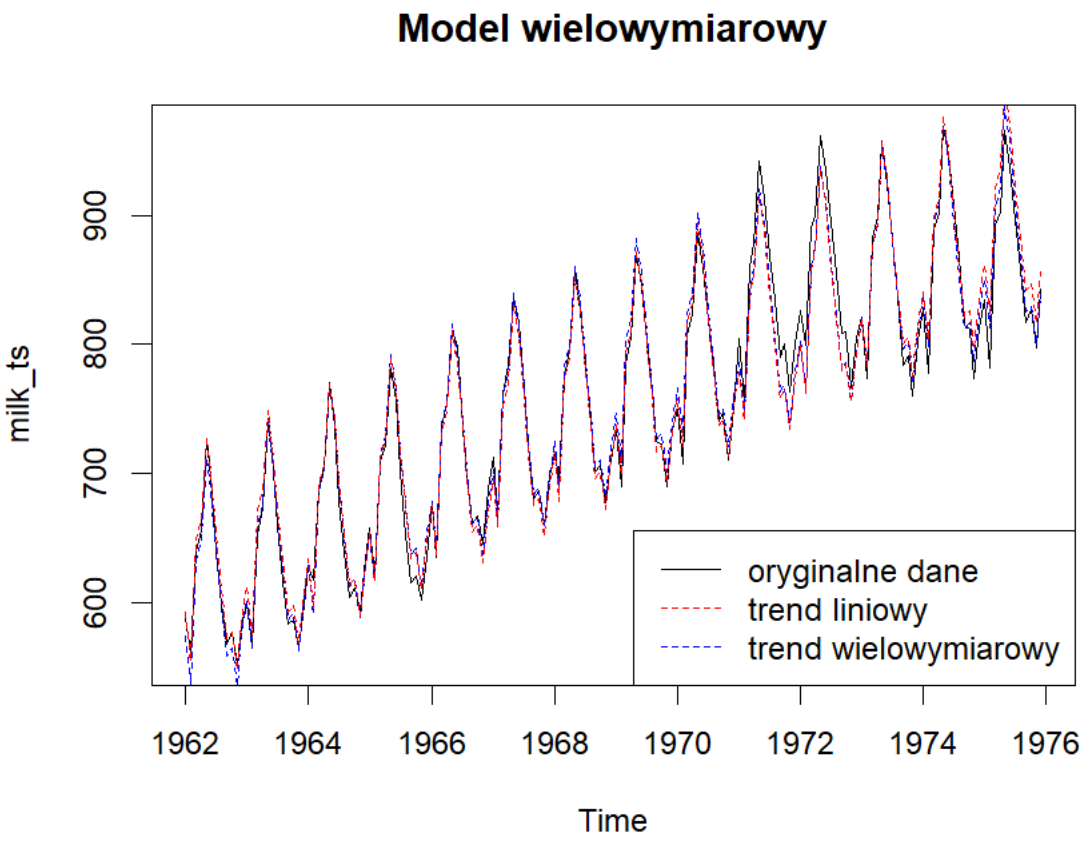

PACF
0.6
0.2
-0.2
0
5
10
15
20
25
30
35
Lag

Na pierwszym wykresie widać, że dane są bliższe danym oryginalnym. Wykres reszt bardziej przypomina szereg stacjonarny. Wykresy ACF i PCAF również wyglądają lepiej.

Trend wielowymiarowy

Do stworzenia trendu wielowymiarowego zastosujemy trend kwadratowy zachowując składową sezonową.

```
> milk.tlsm.2=tslm(milk_ts~season+trend+I(trend^2))
> summary(milk.tlsm.2)

Call:
tslm(formula = milk_ts ~ season + trend + I(trend^2))

Residuals:
    Min      1Q  Median      3Q     Max 
-30.354  -8.537  -2.532   6.888  33.480 

Coefficients:
              Estimate Std. Error t value Pr(>|t|)    
(Intercept)  5.722e+02  4.745e+00 120.609  < 2e-16 ***
season2     -3.955e+01  5.222e+00  -7.574 3.13e-12 ***
season3      5.290e+01  5.223e+00  10.130  < 2e-16 ***
season4      6.772e+01  5.223e+00  12.967  < 2e-16 ***
season5      1.287e+02  5.223e+00  24.640  < 2e-16 ***
season6      1.003e+02  5.224e+00  19.205  < 2e-16 ***
season7      5.052e+01  5.224e+00   9.670  < 2e-16 ***
season8      8.227e+00  5.225e+00   1.575    0.117    
season9     -3.434e+01  5.225e+00  -6.572 7.24e-10 ***
season10    -3.083e+01  5.226e+00  -5.900 2.23e-08 ***
season11    -6.182e+01  5.227e+00 -11.826  < 2e-16 ***
season12    -2.508e+01  5.228e+00  -4.797 3.78e-06 ***
trend        2.378e+00  8.844e-02  26.885  < 2e-16 ***
I(trend^2)  -3.845e-03  5.068e-04  -7.588 2.89e-12 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 13.82 on 154 degrees of freedom
Multiple R-squared:  0.9831,	Adjusted R-squared:  0.9817 
F-statistic:   691 on 13 and 154 DF,  p-value: < 2.2e-16


> plot(milk_ts,main="Model wielowymiarowy")
> lines(fitted(milk.tlsm.2), col="blue",lty=2)
> lines(fitted(milk.tlsm.trend.sez), col="red",lty=2)
> legend("bottomright",legend=c("oryginalne dane","trend liniowy", "trend wielowy
miarowy"),
+        col=c("black","red", "blue"),lty=c(1,2,2))
```

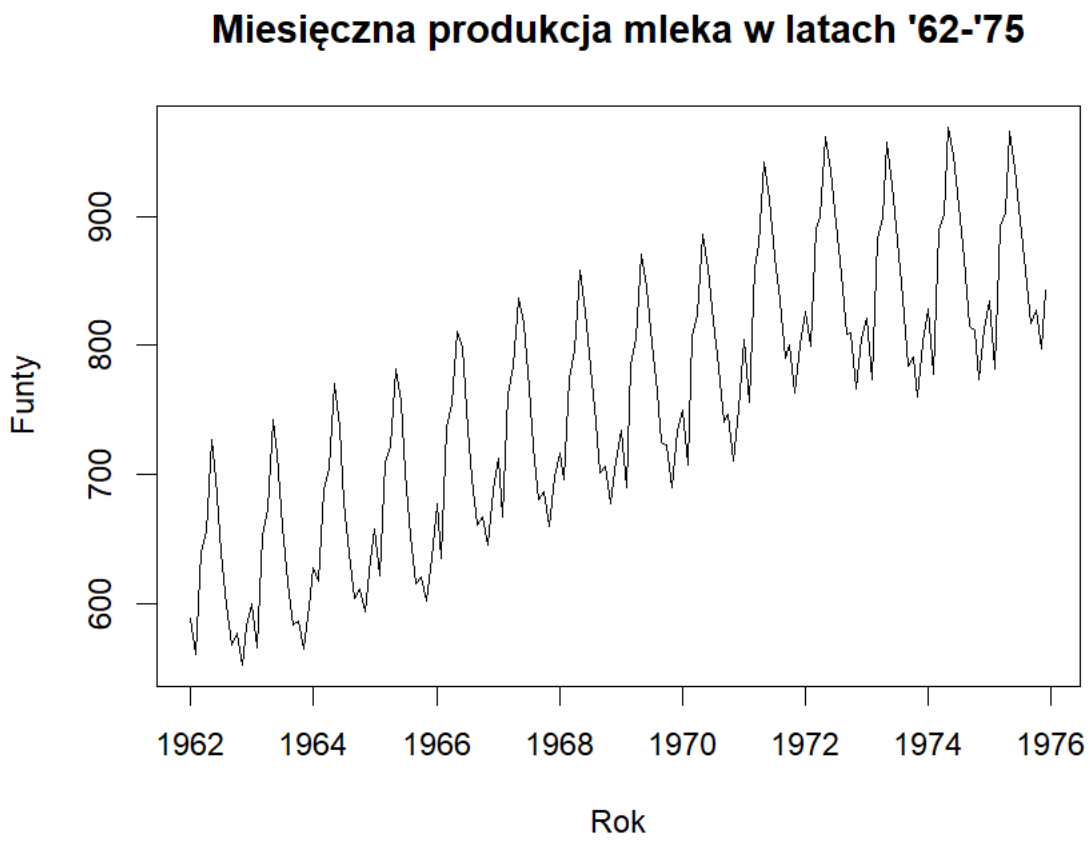


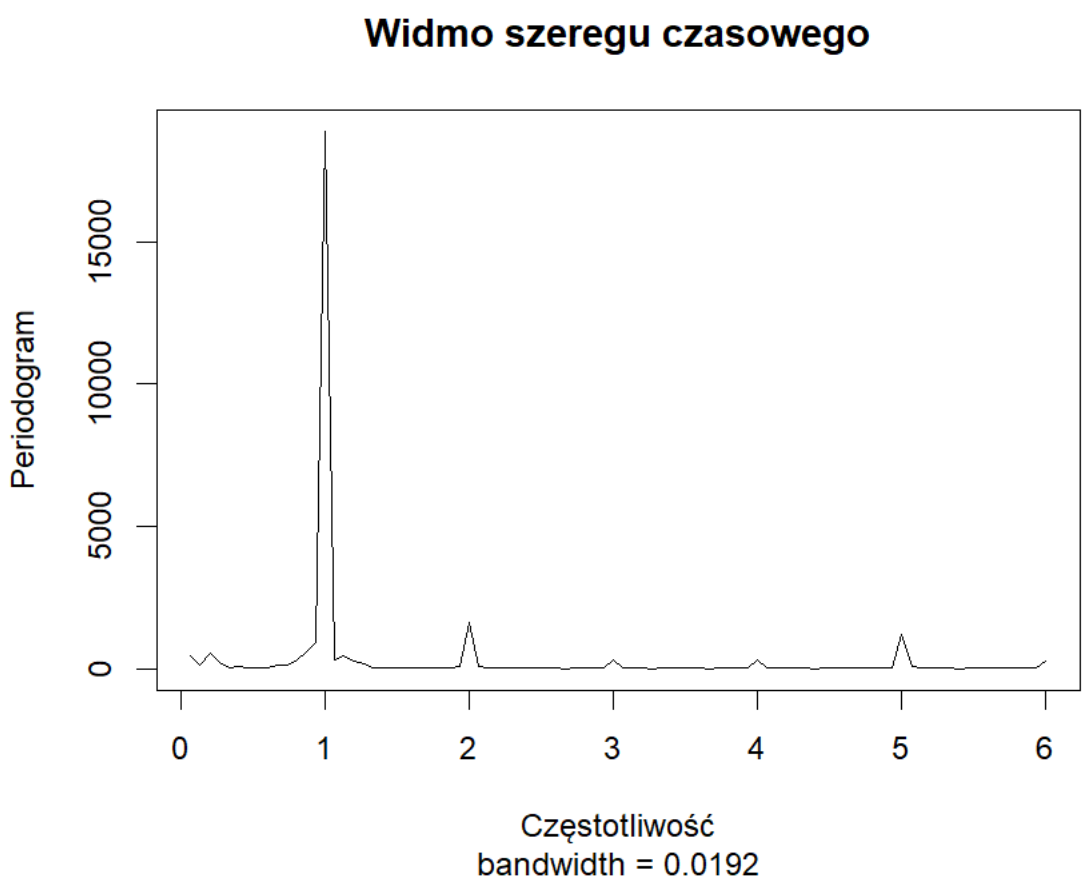
Estymator trendu liniowego wynosi: m_t =0,057 + 2,38t – 0,004t^2 prawie wszystkie współczynniki sezonowe są są istotne statystycznie; R^2=0,98. Model jest bardzo podobny do modelu trendu liniowego z sezonowością.

7. Analiza widmowa

Do analizy widmowej wykorzystamy funkcję *spectrum*. Wartości nie będą skalowane logarytmicznie. Przypomnijmy jak wygląda surowy wykres naszego zbioru danych.

```
> plot(milk_ts, main = "Miesięczna produkcja mleka w latach '62-'75",
+      xlab = "Rok",ylab="Funty")
> spec=spectrum(milk_ts, log="no", main="Widmo szeregu czasowego",
+               xlab="Częstotliwość", ylab="Periodogram")
```





Najwyższy pik mamy przy częstotliwości równej 1, potem leciutkie pagórki przy 2 i 5. Częstotliwość równa 1 to roczna sezonowość, a górki przy częstotliwości równej 2 i 5 znaczy, że istnieje w danych cykl półroczny i co 2,5 miesiąca. Istotnym wydaje się być jednak tylko sezonowość roczna.

8. Dopasowanie modeli z estymacją parametrów

W celu dopasowania modelu i wyestymowania jego parametrów zastosujmy opcję automatycznego wyboru optymalnego rzędu modelu AR na podstawie kryterium AIC.

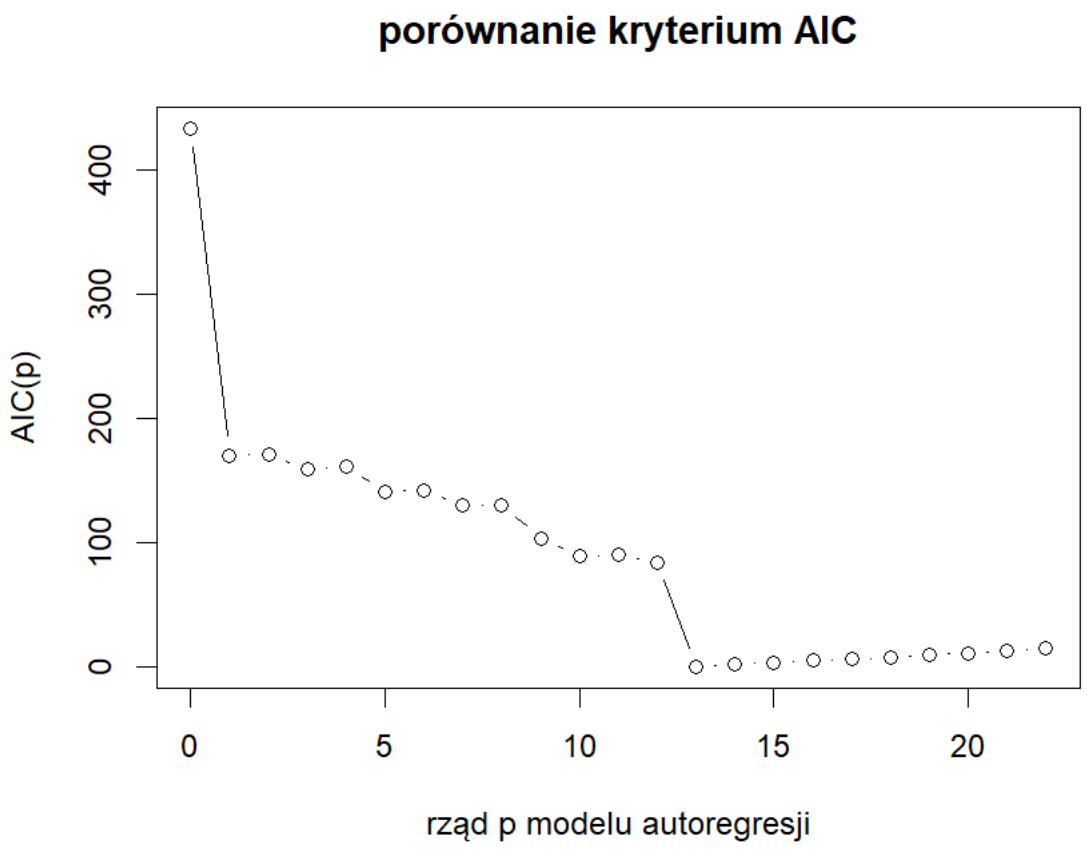
```
> milk.optym.aic=ar(milk_ts,aic=TRUE)
> print(milk.optym.aic)

Call:
ar(x = milk_ts, aic = TRUE)

Coefficients:
      1        2        3        4        5        6        7        8
 0.9144   0.0579  -0.1188  -0.0744   0.1086  -0.0378   0.0543  -0.0848
      9       10       11       12       13
 0.0464   0.0527  -0.0310   0.7114  -0.6318

Order selected 13  sigma^2 estimated as  731.7
```

Model automatyczny wskazał na AR(13). Wypiszmy wartości wartości kryterium AIC dla kolejnych rzędów p.

```
> milk.aic=milk.optym.aic$aic
> print(milk.aic)
         0          1          2          3          4          5          6 
434.254102 170.095711 170.974815 159.077566 160.903114 140.914337 141.774566 
         7          8          9         10         11         12         13 
129.700332 130.276195 103.130048  88.785828  90.123394  83.604060   0.000000 
        14         15         16         17         18         19         20 
  1.956789   3.061295   4.790998   5.521144   7.285686   8.927169  10.785504 
        21         22 
 12.569653  14.115631 
```

Rzeczywiście 13 rząd wykazuje najmniejszą różnice pomiędzy kryterium AIC i wartością minimalną spośród sprawdzanych. Możemy uznać, że szereg jest realizacją modelu AR(13). Skorzystajmy z wykresu i zwizualizujmy powyższe wartości.

```
> plot(as.numeric(names(milk.aic)),milk.aic,xlab= "rząd p modelu autoregresji",yl
ab="AIC(p)", main="porównanie kryterium AIC", type="b")
```



Przedstawiony wykres potwierdza wcześniej wysnute wnioski. Dokładnie widać szybki spadek różnic do rzędu 13, a potem delikatny wzrost wartości.

Sprawdźmy dodatkowo dopasowanie modelu ARIMA do danych. Skorzystamy z funkcji *autoarima*.

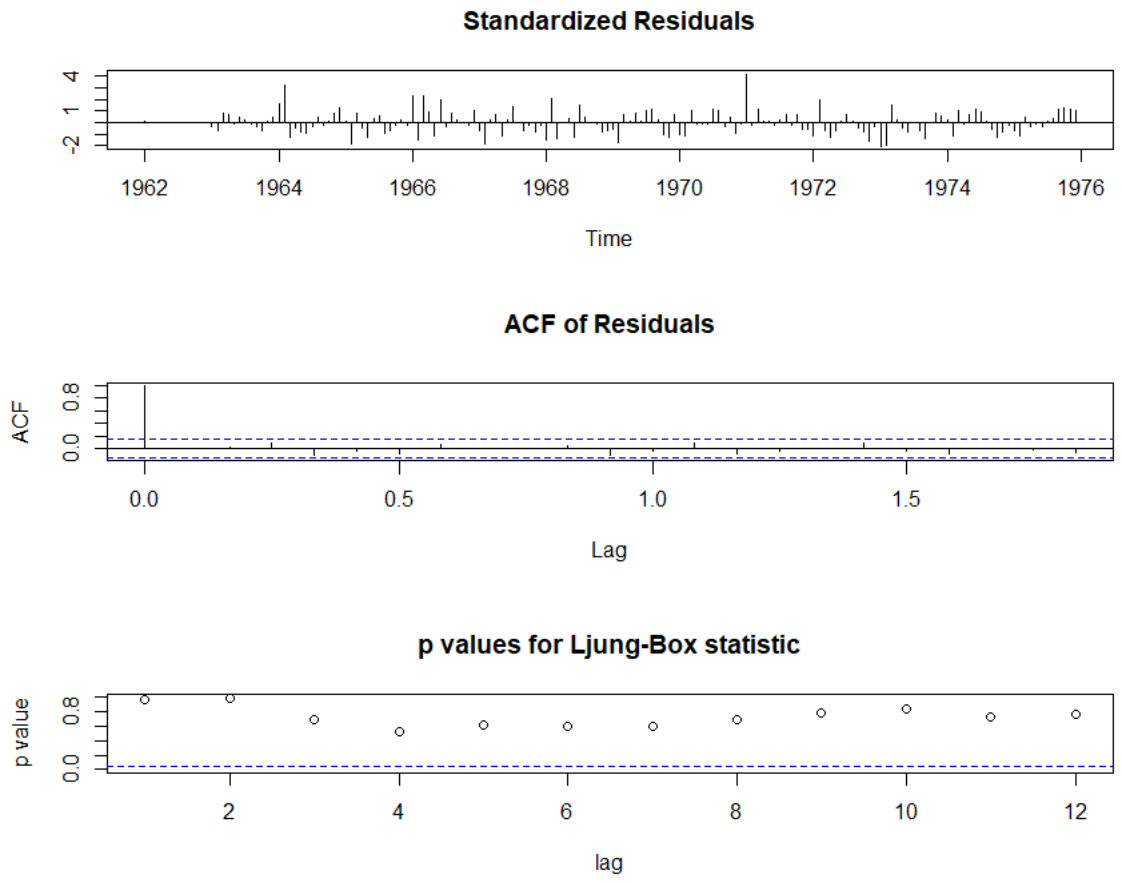
```
Series: milk_ts
ARIMA(0,1,1)(0,1,1)[12]
Box Cox transformation: lambda= 0.7847685

Coefficients:
          ma1     sma1
      -0.2137  -0.6428
s.e.   0.0764   0.0623

sigma^2 = 3.087:  log likelihood = -309.42
AIC=624.85   AICc=625   BIC=633.98

Training set error measures:
                     ME     RMSE      MAE          MPE      MAPE    MASE
Training set 0.01010918 6.963606 5.186082 -0.004954532 0.6909551 0.24361
                     ACF1
Training set 0.0004763403
```

Z transformacji Boxa-Coxa otrzymaliśmy wartość lambdy taką samą jak poprzednio – optymalną. Widzimy, że dostaliśmy model ARIMA(0,1,1)×(0,1,1)[12]. Model ma jedno opóźnienie AR, jeden składnik MA, a także sezonowe składniki AR(1) i MA(1). Błąd RMSE wynosi 6,96, błąd MAPE jest równy około 0,69% - bardzo dobry wynik. MASE jest mniejsze od 1 co oznacza, że model jest lepszy niż prognoza naiwna. Reszty nie wykazują autokorelacji (ACF1 bliskie 0), co potwierdza właściwą specyfikację modelu.

9. Wykonanie badań diagnostycznych

Do wykonania badań diagnostycznych wykorzystamy polecenie *tsdiag*.

```
> tsdiag(arima, gof.lag = 12)
```



Standaryzowane reszty na pierwszym panelu rozkładają się w sposób losowy wokół zera, nie mając przy tym żadnych wyraźnych trendów lub sezonowości.
Wykres funkcji autokorelacji (ACF) potwierdza brak istotnej korelacji pomiędzy resztami w różnych opóźnieniach. Wszystkie wartości autokorelacji mieszczą się w przedziale ufności, co świadczy o losowości reszt.
Test Ljunga-Boxa potwierdza właściwości reszt. Wysokie p-wartości dla wszystkich testowanych opóźnień wskazują na brak podstaw do odrzucenia hipotezy o braku autokorelacji w resztach. Oznacza to, że reszty modelu zachowują się jak biały szum, co potwierdza trafność dobranego modelu ARIMA.
Dodatkowo w celu sprawdzenia dokładny p-wartości wykorzystamy test Liunga-Boxa.

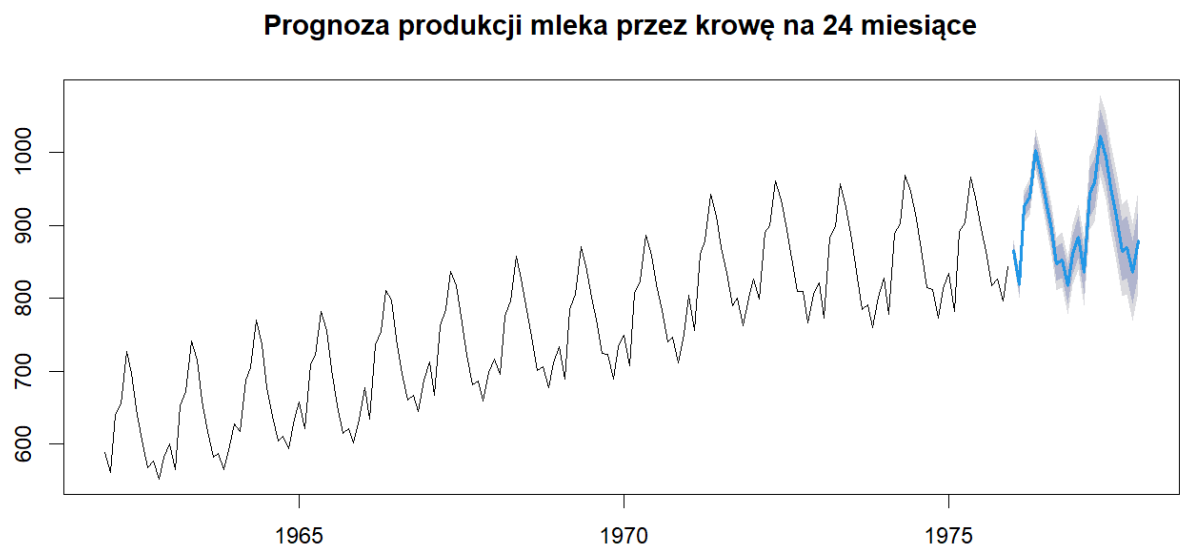
```
> arima.reszty=residuals(arima)
> Box.test(arima.reszty,lag=1,type="Ljung-Box")

        Box-Ljung test

data:  arima.reszty
X-squared = 0.00099459, df = 1, p-value = 0.9748

>
> arima.reszty=residuals(arima)
> Box.test(arima.reszty,lag=12,type="Ljung-Box")

        Box-Ljung test

data:  arima.reszty
X-squared = 8.3523, df = 12, p-value = 0.757

>
> arima.reszty=residuals(arima)
> Box.test(arima.reszty,lag=24,type="Ljung-Box")

        Box-Ljung test

data:  arima.reszty
X-squared = 16.854, df = 24, p-value = 0.8548
```

Dla wszystkich trzech testów p-wartości są większe od 0,05. Potwierdza to, brak autokorelacji i prawidłowym doborze modelu ARIMA.

10. Prognozowanie

Wyznaczmy prognozy dla kolejnych 24 miesięcy, wykorzystując funkcję forecast().

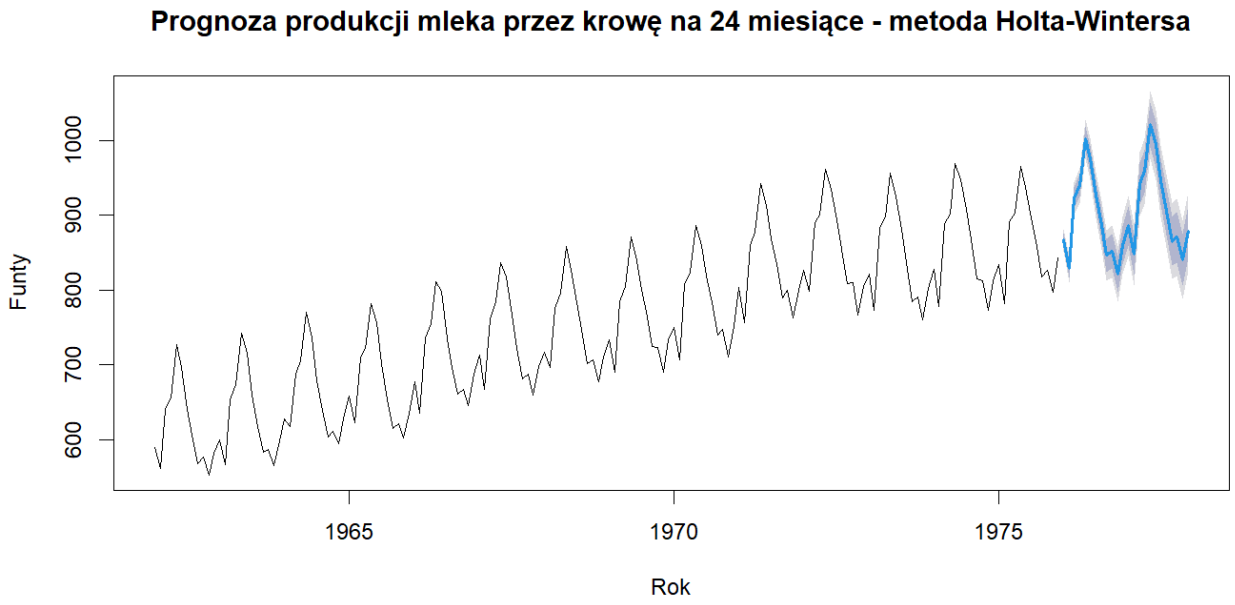
```
> arima.model.prog=forecast(arima,h=24)
> arima.model.prog
         Point Forecast    Lo 80     Hi 80    Lo 95     Hi 95
Jan 1976       865.6277 855.9856  875.2929 850.8909  880.4187
Feb 1976       818.1677 806.0549  830.3191 799.6586  836.7673
Mar 1976       925.7462 911.1243  940.4179 903.4043  948.2047
Apr 1976       939.1642 922.5967  955.7949 913.8521  964.6240
May 1976      1003.0036 984.4712 1021.6101 974.6909 1031.4895
Jun 1976       975.2939 955.3047  995.3716 944.7593 1006.0356
Jul 1976       933.2591 912.0205  954.6024 900.8202  965.9427
Aug 1976       893.1496 870.7780  915.6425 858.9849  927.5980
Sep 1976       846.7161 823.3586  870.2131 811.0512  882.7074
Oct 1976       851.8869 827.3164  876.6109 814.3725  889.7603
Nov 1976       817.3754 791.9080  843.0147 778.4971  856.6559
Dec 1976       860.0181 833.1853  887.0323 819.0555  901.4051
Jan 1977       883.7765 854.6172  913.1445 839.2669  928.7740
Feb 1977       836.0994 805.5498  866.8912 789.4776  883.2878
Mar 1977       944.1571 911.0955  977.4697 893.6972  995.2043
Apr 1977       957.6318 922.8554  992.6823 904.5587 1011.3458
May 1977      1021.7325 984.8983 1058.8548 965.5181 1078.6208
Jun 1977       993.9110 955.8092 1032.3297 935.7702 1052.7934
Jul 1977       951.7019 912.5414  991.2123 891.9557 1012.2668
Aug 1977       911.4202 871.2736  951.9511 850.1802  973.5592
Sep 1977       864.7795 823.7976  906.1840 802.2780  928.2693
Oct 1977       869.9738 827.6817  912.7133 805.4791  935.5151
Nov 1977       835.3034 792.1788  878.9127 769.5513  902.1895
Dec 1977       878.1418 833.3527  923.4281 809.8492  947.5977
> plot(arima.model.prog,main = "Prognoza produkcji mleka przez krowę na 24 miesią
ce")
```



Na pierwszy rzut oka prognoza wygląda prawdopodobnie. Model ARIMA wyraźnie odtwarza sezonowość danych, niemalże z taką samą amplitudą jak dane historyczne. Przedziały ufności są dość szerokie w pikach i dolinach prognozy.

Dodatkowo sprawdźmy prognozę metodą Holta-Winteresa.

```
> plot(arima.model.prog,main = "Prognoza produkcji mleka przez krowę na 24 miesiące")
> milk.hw=hw(milk_ts, seasonal = "additive", h=24)
> plot(milk.hw, main = "Prognoza produkcji mleka przez krowę na 24 miesiące - metoda Holta
-Wintersa",
+              xlab = "Rok", ylab = "Funty")
> milk.hw
         Point Forecast    Lo 80     Hi 80    Lo 95     Hi 95
Jan 1976       866.9927 857.3511  876.6344 852.2471  881.7384
Feb 1976       828.6343 816.7179  840.5508 810.4097  846.8590
Mar 1976       923.1375 909.3152  936.9599 901.9981  944.2770
Apr 1976       939.7290 924.2330  955.2250 916.0300  963.4280
May 1976      1002.2440 985.2379 1019.2501 976.2354 1028.2525
Jun 1976       975.4559 957.0630  993.8489 947.3263 1003.5856
Jul 1976       927.2905 907.6078  946.9733 897.1883  957.3927
Aug 1976       886.9869 866.0935  907.8803 855.0333  918.9406
Sep 1976       846.4018 824.3638  868.4397 812.6977  880.1058
Oct 1976       851.6340 828.5078  874.7601 816.2656  887.0024
Nov 1976       821.4789 797.3131  845.6446 784.5206  858.4371
Dec 1976       859.7003 834.5377  884.8629 821.2174  898.1832
Jan 1977       885.9358 859.8137  912.0578 845.9855  925.8860
Feb 1977       847.5773 820.5299  874.6248 806.2118  888.9429
Mar 1977       942.0805 914.1380  970.0230 899.3462  984.8149
Apr 1977       958.6720 929.8620  987.4820 914.6110 1002.7330
May 1977      1021.1870 991.5347 1050.8393 975.8377 1066.5363
Jun 1977       994.3990 963.9274 1024.8705 947.7967 1041.0012
Jul 1977       946.2335 914.9639  977.5031 898.4108  994.0563
Aug 1977       905.9299 873.8820  937.9779 856.9168  954.9431
Sep 1977       865.3448 832.5367  898.1529 815.1691  915.5204
Oct 1977       870.5770 837.0258  904.1282 819.2648  921.8891
Nov 1977       840.4219 806.1435  874.7003 787.9976  892.8462
Dec 1977       878.6433 843.6526  913.6340 825.1297  932.1570
```

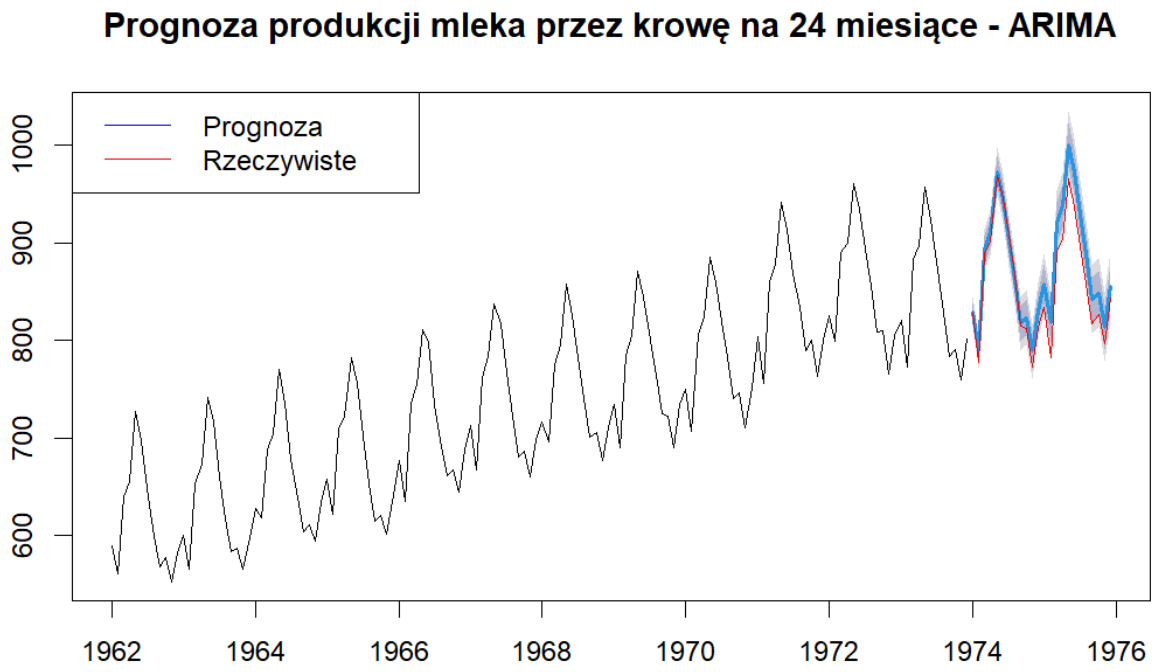


Wyniki są bardzo podobne do prognozy z modelem ARIMA. Przedziały ufności w prognozie Holta-Winteresa są delikatnie węższe – praktycznie nie zauważalnie, co można uznać za pomijalną różnice.

11. Ocena dokładności prognoz

Podzielny nasz zbiór na uczący oraz testowy i oceńmy dokładność prognoz.

```
> n <- length(milk_ts)
> train_size <- n-24
> test_size <- n - train_size
> train_data <- window(milk_ts, end = time(milk_ts)[train_size])
> test_data <- window(milk_ts, start = time(milk_ts)[train_size + 1])
> arima_model <- auto.arima(train_data, lambda = "auto")
> forecast_result <- forecast(arima_model, h = test_size)
>
> plot(forecast_result)
> lines(test_data, col = "red")
> legend("topleft", legend = c("Prognoza", "Rzeczywiste"),
+        col = c("blue", "red"), lty = 1)
```

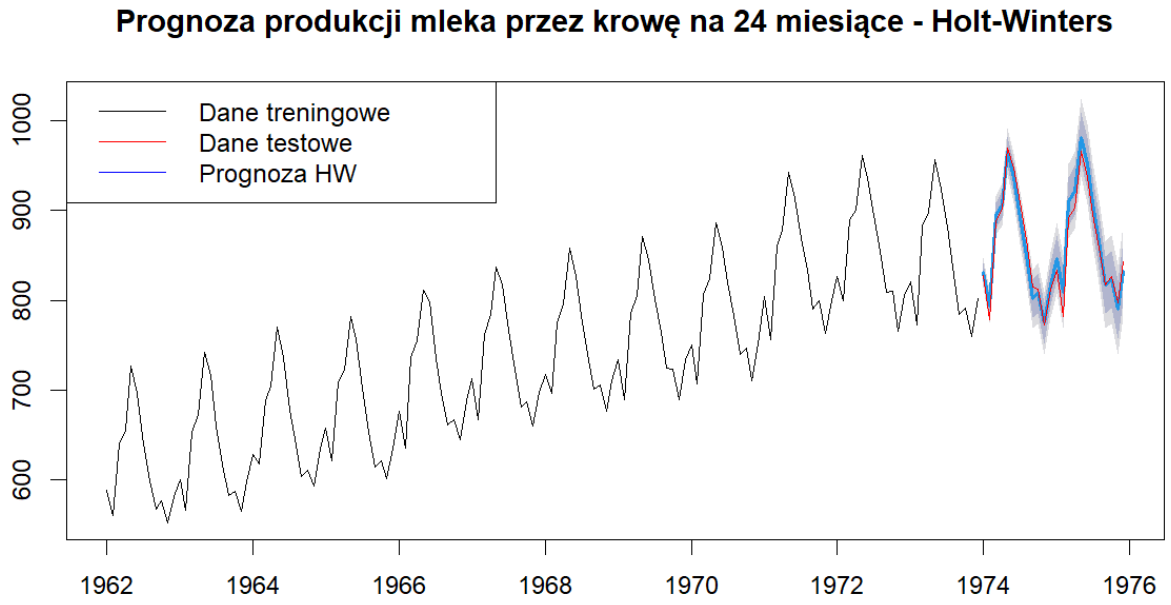


Wizualnie prognoza na pierwsze 12 miesięcy jest praktycznie w punkt. Prognoza na dalszy rok jest zdecydowanie zawyżona. Zahamowanie wzrostu trendu nie zostało przewidziane. Wywołajmy funkcję *accuracy*.

```
> test_accuracy <- accuracy(forecast_result, test_data)
> print(test_accuracy)
                       ME      RMSE       MAE          MPE      MAPE
Training set   0.08378404  6.982313  5.119781  0.005818984 0.6951676
Test set     -16.67120866 21.376840 17.582704 -1.947785798 2.0493265
                   MASE        ACF1 Theil's U
Training set 0.2243729 0.001391869        NA
Test set     0.7705568 0.878242925 0.4179662
```

Model bardzo dobrze dopasowuje się do danych treningowych z błędem MAPE na poziomie zaledwie 0,7%. Na zbiorze testowym jakość prognoz nieco spada, ale nadal pozostaje dobra - błąd MAPE wynosi 2% i model jest lepszy niż prognoza naiwna. Występuje lekkie przeuczenie jednak model zachowuje wartość predykcyjną dla nowych danych. Wykonajmy analogiczną analizę dla prognozy Holta-Wintersa.

```
> hw_model <- HoltWinters(train_data)
> hw_forecast <- forecast(hw_model, h = test_size)
>
> plot(hw_forecast, main = "Prognoza produkcji mleka przez krowę na 24 miesiące
- Holt-Winters")
> lines(test_data, col = "red")
> lines(train_data, col = "black")
> legend("topleft", legend = c("Dane treningowe", "Dane testowe", "Prognoza H
W"),
+        col = c("black", "red", "blue"), lty = 1)
```



Prognoza Holta-Wintersa wygląda prawie identycznie do prognozy z modelem ARIMA. Z wykresu wydaje się, że mógł lepiej przewidzieć wartości w 2 roku prognozy. Sprawdźmy wyniki funkcji *acurracy*.

```
> hw_test_accuracy <- accuracy(hw_forecast, test_data)
> print(hw_test_accuracy)
                     ME      RMSE       MAE         MPE     MAPE      MASE
Training set  0.7804813  8.270668  6.332456  0.09482216 0.857209 0.2775180
Test set     -3.0978983 12.242822 10.274225 -0.36287105 1.193332 0.4502649
                  ACF1 Theil's U
Training set 0.2024582        NA
Test set     0.8175668 0.2397922
```

Model Holt-Winters wykazuje idealną wręcz generalizację na nowe dane, z błędem MAPE na poziomie zaledwie 1,19% na zbiorze testowym, co jest znacząco lepsze niż w przypadku modelu ARIMA. Różnica między błędami treningowymi i testowymi jest minimalna, co wskazuje na brak

przeuczenia. Model jest ponad dwukrotnie lepszy niż prognoza naiwna i generuje bardziej stabilne, mniej obciążone prognozy niż poprzednio testowane rozwiązanie ARIMA.